Plan de gestión, análisis, diseño y memoria del proyecto Sota, Caballo y Rey.

Grado en Ingeniería Informática (Plan 439)

Proyecto Software (30226)
Curso: 2024-2025

Javier Salafranca Pradilla (816341)
Óscar Ventas Fernández (839949)
Diego Viñuales Bello (874280)
Javier Domingo García (871550)
Alberto Serrano Fernández (817959)
José Manuel Vivas Pozuelo (870705)
Alejandro Villacampa Cabo (870379)
Jorge Soria Romeo (872016)

Fecha de entrega: 26/05/2025
Grupo: Grace Hopper E.01
Enlace de la organización en GitHub:
`https://github.com/UNIZAR-30226-2025-01`

Escuela de Ingeniería y Arquitectura
Universidad Zaragoza

Universidad de Zaragoza - Escuela de Ingeniería y Arquitectura

Índice

1. Introducción **4**

2. Organización del proyecto **5**

3. Plan de gestión del proyecto **6**
3.1. Inicio del proyecto . . . 6
3.1.1. Recursos . . . 6
3.1.2. Riesgos y estrategias para mitigarlos . . . 6
3.1.3. Formación inicial . . . 7
3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto . . . 8
3.2.1. Responsables de control de calidad . . . 8
3.2.2. Convenciones de nombrado . . . 8
3.2.3. Estructura de repositorios . . . 10
3.2.4. Herramientas necesarias para la construcción del software . . . 10
3.2.5. Procedimientos de cambios en el repositorio . . . 11
3.2.6. Construcción del software . . . 12
3.2.7. Despliegue del software . . . 12
3.2.8. Documentación técnica del proyecto . . . 12
3.2.9. Medidas de control de calidad aplicadas . . . 13
3.2.10. Entrega de resultados al cliente . . . 15
3.3. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento . . . 15

4. Análisis y diseño del sistema **18**
4.1. Análisis de requisitos . . . 18
4.2. Diseño del sistema . . . 20
4.2.1. Diagrama de módulos Backend . . . 21
4.2.2. Diagrama de módulos frontend móvil . . . 24
4.2.3. Diagrama de módulos Frontend escritorio . . . 27
4.2.4. Diagrama de despliegue. . . . 32
4.2.5. Mapeo entre vistas. . . . 33
4.2.6. Mapeo con los requisitos. . . . 34
4.2.7. Tecnologías elegidas . . . 35
4.2.8. Otros aspectos técnicos de interés . . . 36

5. Memoria del proyecto **38**
5.1. Inicio del proyecto . . . 38
5.2. Ejecución y control del proyecto . . . 38
5.2.1. Reparto de trabajo entre los miembros del equipo . . . 39
5.2.2. Comunicación interna . . . 39
5.2.3. Medición del progreso del proyecto y rastreo de cosas completas e incompletas . . . 39
5.2.4. Ajustes realizados por divergencias en el calendario . . . 40
5.2.5. Ajuste en tecnologías y herramientas empleadas . . . 40
5.2.6. Control de versiones, construcción y despliegue . . . 40
5.2.7. Pruebas del software . . . 40

5.3. Cierre del proyecto 44

6. Conclusiones **47**

A. Anexo I: Riesgos **48**

B. Anexo II: Presentación final **49**
B.1. Diapositivas de la presentación final 49
B.2. Ideas de la teoría tenidas en cuenta 54
B.2.1. Conoce y conecta con la audiencia 54
B.2.2. La gran idea 54
B.2.3. Explicaciones 54
B.2.4. Visualización 55
B.2.5. Transparencias 55
B.2.6. Presentación 55

C. Anexo III: Resultados revisión intermedia **56**

1. Introducción

El presente documento ofrece una visión detallada del desarrollo y gestión de un proyecto de software consistente en un juego multijugador de cartas con las reglas del **Guiñote** español, disponible para dispositivos móviles y equipos de escritorio. Esta aplicación ofrecerá interfaces adaptadas a los dispositivos en los que se ejecutará y varias funcionalidades sociales que harán la experiencia de juego más interactiva y enriquecedora. Nuestro objetivo es desarrollar de forma eficaz un juego entretenido y disfrutable del que sea sencillo hacerse a su mecánica y en el que la interacción del jugador con la aplicación sea lo más satisfactoria posible.

El documento se organiza en seis capítulos principales y tres anexos, de forma que se pueda acceder de forma más directa a la información que más interese al lector. En primer lugar, se presenta el capítulo *2. Organización del proyecto*, donde se detalla quiénes integran el proyecto y cómo se distribuyen las responsabilidades.

Posteriormente, en el capítulo *3. Plan de gestión del proyecto* se detallan los recursos utilizados, los riesgos identificados, las estrategias para su mitigación y la organización general del desarrollo y la calidad del software. A continuación, en el capítulo *4. Análisis y diseño del sistema*, se profundiza en los requisitos del proyecto, el diseño técnico, las decisiones arquitectónicas tomadas y las tecnologías empleadas. El capítulo *5. Memoria del proyecto* describe cómo se ha llevado a cabo el desarrollo real, la ejecución del trabajo por parte del equipo, el control interno del proyecto y los ajustes realizados a lo largo del proceso. Finalmente, en el capítulo *6. Conclusiones*, se evalúan los resultados del proyecto y se extraen valoraciones finales sobre el desarrollo realizado. Adicionalmente, se incluye el *A. Anexo I: Riesgos*, que recoge un análisis detallado de los problemas potenciales y reales que han podido afectar al desarrollo del proyecto. También se incluye el *B. Anexo II: Presentación*, con la presentación al público del proyecto; y el *C. Anexo III: Resultados revisión intermedia* que recoge el resultado de la autoevaluación intermedia.

Como ya se ha dicho, la aplicación permite jugar de forma completamente online partidas de guiñote de forma síncrona con otras personas a través de un sistema de emparejamiento por puntos de habilidad, así como configurar sesiones de juego privadas con amigos y chatear con ellos. Las partidas podrán ser de 1 contra 1 o entre parejas, de forma que puedan encontrarse 4 personas en la misma sesión. El juego permite, además, que el usuario pueda personalizar sus barajas o tapetes de forma que su experiencia sea todavía más única. Estos ítems se obtendrán mediante el sistema de progreso por rangos del juego.

Se entregarán dos paquetes de software, uno por cada tipo de dispositivo para los que la aplicación está desarrollada. En primer lugar, para dispositivos móviles se entregará un paquete *APK*, el cual puede ser instalado sin problemas en dispositivos *Android* compatibles. Por otro lado, en el caso de la versión de escritorio, lo que se suministrará será un ejecutable para *Linux*, plataforma ampliamente utilizada en entornos de desarrollo y con creciente presencia en el ámbito de juegos para ordenador.

2. Organización del proyecto

El equipo de este proyecto está formado por **Javier Salafranca** (**Javier S.**), **Javier Domingo** (**Javier D.**), **Óscar Ventas**, **Diego Viñuales**, **Jorge Soria**, **José Manuel Vivas**, **Alberto Serrano** y **Alejandro Villacampa**. La dirección del proyecto recae en **Javier S.**, cuya función principal será supervisar todas las fases de trabajo y tomar las decisiones finales, asegurando así el cumplimiento de objetivos y plazos.

En cuanto a las responsabilidades específicas:

- **Óscar** se encarga de la comunicación directa con el profesor (nuestro cliente) a través del correo electrónico, resolviendo dudas y trasladando la información clave al resto del equipo.
- **José** asume la organización y el mantenimiento de los repositorios en GitHub, definiendo la estructura del proyecto y garantizando un flujo de trabajo ordenado en la plataforma.
- **Javier S.** tiene la labor de recopilar periódicamente las horas de trabajo que cada integrante anota en un bot de *Discord*, permitiendo así llevar un control adecuado del progreso y la dedicación.

Además, el desarrollo se ha dividido en tres frentes principales:

- Frontend de escritorio (a cargo de **José**, **Javier S.** y **Diego**)
- Frontend móvil (asignado a **Javier D.**, **Óscar** y **Alberto**)
- Backend (en manos de **Jorge** y **Alejandro**)

Pese a esta división, es frecuente la comunicación entre los diferentes frentes para definir aspectos estéticos compartidos o solicitar funcionalidades específicas, reforzando la coherencia y la calidad global del proyecto.

3. Plan de gestión del proyecto

Esta sección describe el enfoque general adoptado para la gestión efectiva del proyecto, cubriendo desde la planificación inicial hasta la implementación práctica de las tareas. Incluye la identificación de recursos clave, estrategias para afrontar riesgos, procedimientos de formación inicial del equipo y el establecimiento de políticas internas que faciliten el éxito del proyecto.

3.1. Inicio del proyecto

La etapa inicial del proyecto es crítica para sentar una base sólida que permita un desarrollo eficiente. Por ello, se comienza con la identificación y asignación adecuada de recursos tecnológicos y del entorno que serán esenciales para todas las fases posteriores, garantizando así la productividad y coordinación del equipo desde el primer momento.

3.1.1. Recursos

Se han identificado y asignado recursos hardware y del entorno para las distintas etapas del proyecto. En la etapa inicial de desarrollo, el equipo encargado del backend empleará contenedores e imágenes de **Docker** para el despliegue del servidor y la base de datos. Las imágenes específicas utilizadas serán `django_container`, `postgres_container` y `redis_container`. En esta fase, también se empleará un servidor síncrono como **Gunicorn**, ya que el equipo se centrará en funcionalidades que solo requieren peticiones HTTP. Además, se crearán los scripts correspondientes para la automatización del despliegue mediante Docker. De esta forma, el equipo encargado del frontend únicamente deberá descargar el código fuente del backend y ejecutar el script encargado del lanzamiento y puesta en marcha de dichos contenedores.

En etapas posteriores, se subirá el código fuente del backend a **un servidor en cloud** (p.e. *OVH Cloud*) donde se lanzará con Docker nuestro servidor y base de datos. De esta manera, el frontend podrá acceder a la API a través de una dirección común. Para gestionar funcionalidades que requieren de comunicación bidireccional (como partidas, chats, etc.), se migrará a un servidor asíncrono como **Daphne**. El equipo encargado del frontend hará uso de **emuladores de Android** (frontend móvil) o del propio entorno de desarrollo (frontend escritorio). En etapas finales será útil probar con **dispositivos físicos reales**.

3.1.2. Riesgos y estrategias para mitigarlos

El equipo coincide en que los riesgos más significativos identificados en el desarrollo del proyecto son:

- **Problemas de organización grupal:** existe la posibilidad de dificultades en la coordinación interna debido a que los miembros del equipo no están habituados

a trabajar en grupos numerosos, lo que podría generar conflictos o retrasos en el avance del proyecto.

- **Desviaciones en la planificación temporal:** podrían surgir retrasos derivados de prácticas en otras asignaturas más extensas de lo previsto, controles académicos imprevistos o errores complejos en el código que demanden un esfuerzo adicional no considerado inicialmente.
- **Inexperiencia del equipo:** el grupo presenta poca experiencia previa en proyectos similares de esta magnitud y complejidad, lo cual podría conducir a dificultades técnicas, mala estimación de esfuerzos, o implementación inadecuada de soluciones técnicas.
- **Dificultades en la paralelización del trabajo:** el reparto ineficiente de tareas podría provocar que algunos integrantes dependan excesivamente del progreso de otros, lo que conlleva bloqueos en el avance individual y colectivo si no se gestiona correctamente la comunicación o se especifican claramente los requisitos de las interfaces.

En definitiva, los riesgos identificados más relevantes están relacionados con la gestión interna del equipo, posibles retrasos en la planificación del proyecto y las consecuencias derivadas de la falta de experiencia en desarrollos similares.

Tras identificar los riesgos, se proponen estas estrategias para mitigarlos:

- **División de tareas:** uso de *issues* específicas, división en equipos backend, frontend móvil y frontend escritorio, y reuniones periódicas para revisión de tareas.
- **Flexibilidad interna:** predisposición del equipo a apoyar puntualmente en otros equipos o tareas atrasadas, facilitada mediante seguimiento semanal del progreso.
- **Comunicación continua:** reuniones semanales, uso activo de Git y comunicación instantánea para mantener informados a todos los miembros.

Para consultar las estimaciones de exposición (probabilidad por impacto) de cada riesgo identificado, véase la *Tabla de exposición a riesgos en el Anexo: Riesgos*.

3.1.3. Formación inicial

Para formarse, tanto en frontend como en backend, se han empleado cursos, tutoriales y documentación oficial disponible en plataformas como *YouTube*, así como en las páginas web de los propios frameworks utilizados. El objetivo ha sido adquirir los conocimientos necesarios para desarrollar de forma autónoma y eficiente cada componente del sistema.

En particular, los recursos principales consultados han sido:

- **Frontend Desktop (Qt):** se ha seguido principalmente la guía oficial de Qt para C++, disponible en su web oficial: `https://wiki.qt.io/Main`.

- **Frontend Móvil (Flutter):** se ha utilizado la documentación oficial de Flutter, incluyendo guías paso a paso y ejemplos de código: `https://docs.flutter.dev/`.
- **Backend (Django):** el equipo ha seguido la guía oficial de Django, en especial el tutorial "Writing your first Django app": `https://docs.djangoproject.com/en/stable/intro/tutorial01/`.

Todos los integrantes del equipo han utilizado tecnologías y frameworks que inicialmente desconocían, por lo que se dedicaron entre una y dos semanas a la autoformación y adaptación a las herramientas seleccionadas, utilizando los recursos anteriores como referencia principal.

3.2. Control de configuraciones, construcción del software y aseguramiento de la calidad del producto

Para garantizar la calidad del producto final y la correcta gestión de los cambios que se realicen durante su desarrollo, se han establecido procedimientos claros y responsables específicos dentro de cada equipo. Esto incluye la definición explícita de roles encargados de supervisar la aplicación de medidas de calidad y la correcta organización del código generado durante el desarrollo.

3.2.1. Responsables de control de calidad

Existe un responsable de control de calidad en cada equipo de desarrollo, encargado de supervisar la creación y continua ejecución de las baterías de tests usadas en control de calidad, así como de asegurar que se sigan correctamente los diversos protocolos para la correcta estructuración y despliegue de esa parte del proyecto.

1. **Frontend Android:** Óscar Ventas
2. **Frontend Desktop:** Javier Salafranca
3. **Backend:** Alejandro Villacampa

3.2.2. Convenciones de nombrado

Las convenciones de nombrado para Android Studio pretenden diferenciar las pantallas de los tests.

Frontend Android Aunque no se ha seguido una guía de estilo oficial de Flutter de forma estricta, sí se han aplicado algunas convenciones de forma consistente para mantener la claridad y coherencia del código. En particular:

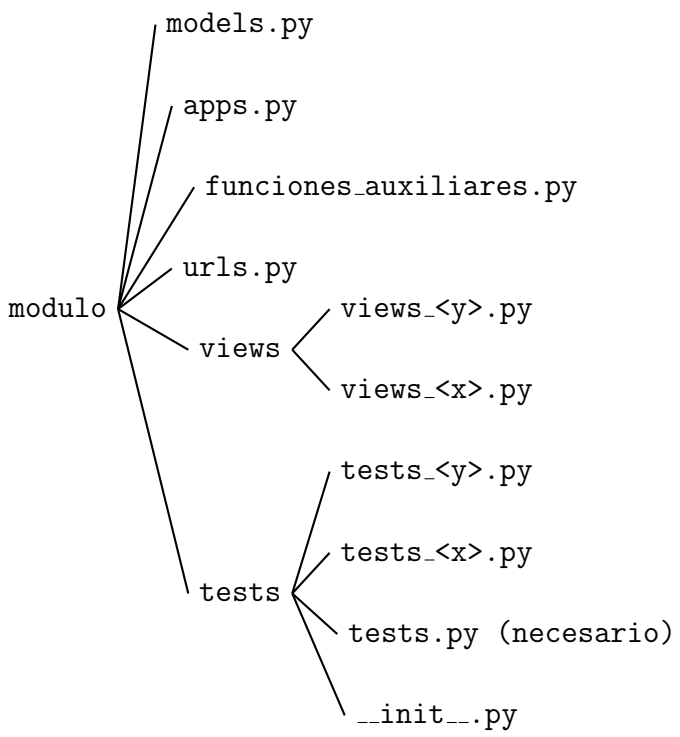
- **Nombres de clases:** se usa `CamelCase` (ej. `HomeScreen`, `GameScreenState`).
- **Ficheros:** se nombra cada archivo en `snake_case` siguiendo el patrón `<nombre_widget>.dart` (ej. `home_screen.dart`, `game_screen.dart`).
- **Widgets:** se organiza el código en widgets reutilizables dentro de la carpeta `/widgets`. Por ejemplo: `custom_button.dart`, `corner_decoration.dart`.
- **Servicios:** los servicios como WebSocket o audio se agrupan en `/services`, siguiendo el patrón `<nombre>_service.dart`.
- **Rutas:** se centralizan en un único fichero llamado `routes.dart` donde se definen las rutas de navegación de la app.

Además, cada pantalla principal implementa su estado en un archivo único, con comentarios de documentación estructurada que describen su funcionalidad.

Frontend Desktop Aunque no se ha seguido una guía de estilo formal documentada, el equipo ha mantenido coherencia en varios aspectos fundamentales:

- Se utiliza **CamelCase** para los nombres de clases (por ejemplo, `EstadoPartida`).
- Los métodos emplean **camelCase**, y las variables también siguen esta convención de forma general.
- Se incluye documentación en formato `Doxygen` mediante bloques `/** ... */`.
- La indentación es clara y uniforme, y se respetan las llaves de apertura y cierre con buena legibilidad.
- Se mantiene una separación lógica entre bloques de código funcional (inicialización, configuración visual, red, etc.).

Backend El esquema de nombrado de archivos en Backend viene fuertemente condicionado por el uso de Django, que requiere el uso de ciertos archivos con nombres determinados. Esta estructura está basada en la forma estándar de organizar aplicaciones dentro de un proyecto Django, tal como se describe en la documentación oficial: `https://docs.djangoproject.com/en/stable/ref/applications/`. Además, se usan estructuras definidas con carpetas dedicadas a funciones y a tests, quedando finalmente esta estructura típica para un módulo concreto del proyecto, además de los archivos externos fijos:



3.2.3. Estructura de repositorios

El proyecto se ha organizado en tres repositorios independientes, cada uno correspondiente a un componente desplegable. Esta separación refleja la arquitectura basada en contenedores diferenciados. Dada la envergadura del proyecto, no se han impuesto controles de acceso estrictos, pero sólo los colaboradores asignados a cada repositorio deben modificar su código.

En caso de que participe personal externo (para pruebas de integración, validaciones puntuales, etc.), se creará una rama aislada que se eliminará tras completar la tarea. Cada repositorio incluye todo lo necesario para poner en funcionamiento su componente: código, scripts de despliegue y la documentación de desarrollo relevante (guías de estilo, especificaciones de API, ejemplos de uso), de modo que facilite la integración y el trabajo coordinado entre los distintos módulos.

3.2.4. Herramientas necesarias para la construcción del software

Frontend Android (Flutter)

- **Flutter SDK** y **Dart CLI** (gestión de paquetes con `pub`).
- **Android Studio** (IDE) con Android SDK/NDK.
- **Gradle**: sistema de construcción que gestiona la compilación y empaquetado del APK.

Frontend Desktop (Qt)

- **Qt Creator**: IDE oficial para desarrollo con Qt.
- **Qt Framework** (módulos `QtCore`, `QtWidgets`, `QtGui`, `QtWebSockets`).
- **Compilador C++** (p. ej. `clang++` o `g++`) con soporte C++17.
- **Sistema de construcción** (`qmake` o `CMake`) para generar Makefiles o proyectos nativos.
- **QPixmap** y clases de `QtGui` para carga y manipulación de imágenes.

Backend

- **Docker** y **Docker Compose** para orquestar contenedores (Django + PostgreSQL + Redis).
- **Python 3.x**, `virtualenv` y `pip` para la instalación de dependencias (`Django`, librerías auxiliares).
- **Shell scripts** para automatizar la construcción de imágenes, migraciones de base de datos y ejecución de tests.

3.2.5. Procedimientos de cambios en el repositorio

Los cambios en el repositorio se gestionan mediante el sistema de incidencias. Se crean incidencias en GitHub asociadas a las entregas relevantes y agrupadas en hitos. Cada tarea queda asignada a un desarrollador del equipo; una vez completada, el propio desarrollador puede autoasignarse las tareas pendientes. Este flujo de trabajo evita conflictos al hacer commits y minimiza la probabilidad de que varios desarrolladores editen simultáneamente el mismo archivo. Además, contribuye a una mayor trazabilidad de los cambios y facilita la revisión de código.

El desarrollador no requiere permiso para subir un commit a su repositorio correspondiente, es él quien tiene la responsabilidad de desarrollar y/o ejecutar los tests relevantes, y del responsable de calidad de su equipo establecer la política para estos y supervisar su correcto cumplimiento. Esto último se realiza principalmente en dos situaciones: Tras commits importantes donde se revisa el código y en las reuniones semanales del grupo. También puede solicitar una reunión privada o mencionar a dicho miembro si hay algo que no supera los umbrales de calidad.

Una incidencia puede encontrarse en tres estados: abierta sin encargado, abierta con encargado o cerrada. Se cierra cuando el desarrollador responsable completa los objetivos descritos y ejecuta los tests pertinentes que demuestran su correcto funcionamiento. Independientemente de quién la registre —miembros del propio equipo o de otros módulos—

todas las incidencias siguen el mismo flujo: se crean en GitHub, se asignan a un desarrollador, se resuelven con los cambios necesarios y se validan mediante la batería de pruebas antes de integrarse en la rama principal.

Ambas posibilidades están respectivamente catalogadas con etiquetas de incidencia y siguen el mismo procedimiento para ser asignadas y completadas que las incidencias de desarrollo creadas por el equipo asignado al repositorio.

3.2.6. Construcción del software

Frontend Android

En Frontend Android, la funcionalidad de actualización del entorno de Android Studio asegura que las versiones de todas las dependencias y emuladores sean uniformes para todos los integrantes del equipo.

Frontend Desktop

En Frontend Desktop, las actualizaciones de dependencias son gestionadas automáticamente por el entorno de QT.

Backend

Para abstraerse de las máquinas de desarrollo y despliegue y asegurar que las versiones del software de desarrollo y despliegue sean idénticas, se hace uso de contenedores de Docker, usando la última versión de Docker; estos vienen definidos por los Dockerfiles presentes en el repositorio y se construyen y despliegan mediante scripts de shell también presentes en los repositorios.

3.2.7. Despliegue del software

El despliegue de software se divide entre la parte local, la cual consiste en uno de los dos clientes, el de Desktop o Android, instalado en el dispositivo del usuario; y los contenedores de Docker desplegados en una máquina la cual actuará como servidor. Este último despliegue en nuestro caso se realizará en una máquina alquilada con Ubuntu, con la ayuda de una serie de scripts de shell que pretenden simplificar el proceso. Para consultar el procedimiento detallado de despliegue del backend, puede accederse a la wiki del repositorio: `https://github.com/UNIZAR-30226-2025-01/guignote-backend/blob/main/README.md`.

3.2.8. Documentación técnica del proyecto

La documentación del proyecto abarca tanto los diagramas UML realizados con draw.io que documentan la arquitectura y se encuentran en este documento, como la información que resulta inmediatamente relevante y tiene que ser fácilmente accesible por los distintos equipos de desarrollo. Esta información reside en el apartado wiki de los distintos repositorios.

En el caso del frontend, principalmente se trata de las guías de estilo que aseguran una estética uniforme entre ambos equipos de desarrollo, lo cual resulta importante, puesto que en la mayoría de los casos, los desarrolladores no cambiarán de un equipo a otro.

En el caso del backend, consiste en la documentación de las distintas peticiones HTTP de las cuales los desarrolladores frontend dependen. La documentación se divide en distintos artículos ordenados según a qué ámbito corresponden las peticiones (estadísticas, lógica de partida...) que contienen especificaciones sobre el formato que se debe emplear para la petición, argumentos necesarios, efectos que tiene la función, valores que devuelve, ejemplos de uso e indicaciones adicionales; como puede ser la explicación del sistema de autenticación.

El principal objetivo de esta estructura es limitar la necesidad que tienen los distintos integrantes del equipo de comunicarse directamente y evitar la necesidad de cualquier tipo de formación sobre el uso de los diversos componentes desarrollados, puesto que las franjas de tiempo de las que cada uno dispone para trabajar son muy irregulares y no tienen porqué coincidir. De esta forma, todas las indicaciones relevantes que un equipo debe dar a otro son recogidas en la documentación y accesibles en cualquier momento. Para esto también resulta útil la estructura de *issues* que permite a un integrante del grupo pedir la realización de distintas tareas a otros y seguir su cumplimiento, minimizando las horas de reunión consumidas.

3.2.9. Medidas de control de calidad aplicadas

Frontend Android

El control de calidad en Android se realiza en un primer nivel mediante tests unitarios ejecutados en el entorno de desarrollo de Android Studio, seguido de un segundo nivel en el cual se ejecuta la aplicación en el emulador integrado en este mismo entorno de desarrollo y se recorren distintos flujos de usuario para asegurarse de que todos los componentes funcionan en todas las situaciones posibles. Una vez estos terminan correctamente. se realiza el mismo proceso en la tercera fase: un dispositivo móvil físico.

Se realizan las pruebas de código en una rama test del repositorio, posteriormente se pasa a una rama develop donde se realizan los tests automáticos y finalmente a main.

Las pruebas involucrando emuladores son realizadas por todos los miembros del equipo de desarrollo de Android tras ser subidas a main para asegurarse completamente del correcto funcionamiento.

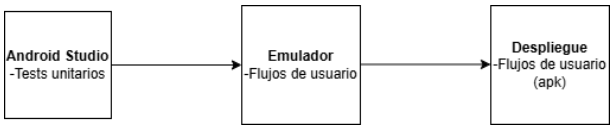


Figura 3.2.1: Despliegue en Android

Frontend Desktop

El control de calidad en Desktop se realiza mediante tests unitarios automáticos de-

sarrollados en QT y posteriormente tests automáticos en QT que prueban el correcto funcionamiento de los distintos elementos de la interfaz. Finalmente, se realizan las mismas pruebas que en Android, desplegando el cliente en las propias máquinas empleadas para el desarrollo y recorriendo también distintos flujos de usuario posibles.

Se realizará también en este cliente un test de integración para comprobar el comportamiento de la aplicación ante entradas que contengan caracteres y símbolos no convencionales y, posiblemente, problemáticos.

Cada integrante de este equipo de desarrollo realiza también revisión de código por pares a aquello que escriben los demás integrantes.

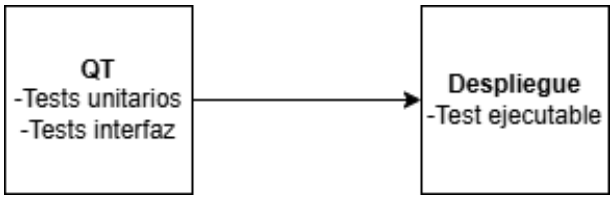


Figura 3.2.2: Despliegue Desktop

Backend

Para los tests implicando las distintas funciones de Backend se emplea la función de tests integrada en Django, junto con las funcionalidades que se pretenden añadir se desarrollan y ejecutan sistemáticamente tests que comprueban el correcto funcionamiento de todas y cada una de las peticiones de las que dependerá el frontend. Nos aseguraremos tanto de que las nuevas peticiones estén correctamente desarrolladas como de que este proceso no haya provocado errores entre las ya desplegadas.

Cada una de estas baterías de test está asociada a los distintos componentes de la aplicación, los cuales están separados en directorios distintos. Se pone especial cuidado en que estas baterías de tests cubran casos problemáticos, véanse peticiones con la base de datos vacía o asegurar que los casos de error salten correctamente. Por la cantidad de estos, se ha elaborado un script de shell que los ejecuta divididos en estos componentes para localizar fácilmente cuál de ellos contiene posibles errores.

Esto se ha realizado mediante un pipeline de despliegue, realizando primero las migraciones (proceso que emplea Django para construir automáticamente la base de datos asociada) en la versión local de Django, y posteriormente se ejecutan los tests en la versión integrada en el contenedor de Docker. Esto se debe a que si se produce un fallo en el proceso de migración del contenedor de Docker terminará con un error difícil de diagnosticar ya que es preferible realizar los tests en el contenedor, puesto que es el entorno real en el que funcionará la aplicación.

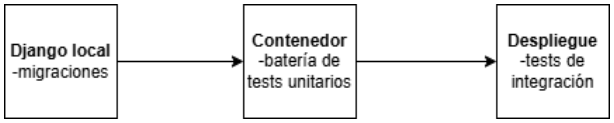


Figura 3.2.3: Despliegue Backend

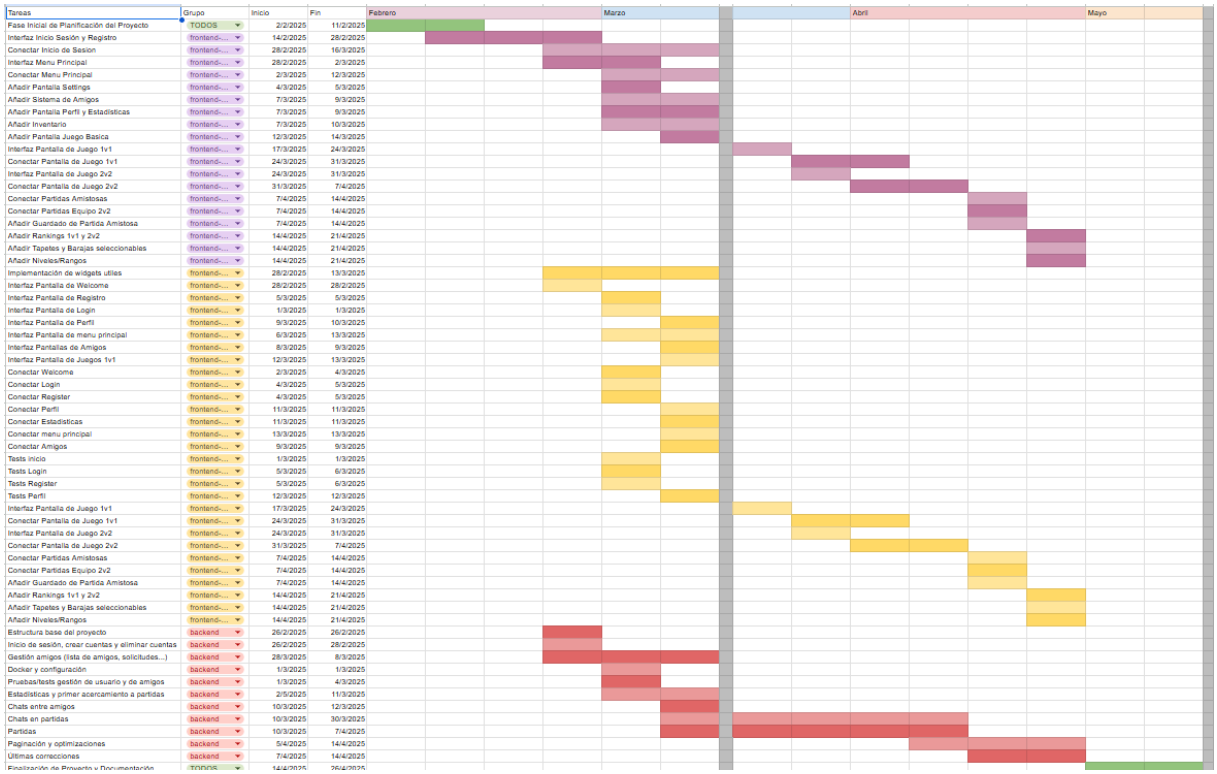
3.2.10. Entrega de resultados al cliente

La entrega de resultados al cliente consistirá en los ejecutables de los dos clientes, un .exe y un .deb en caso del cliente Desktop y un APK en caso del cliente Android. En caso del Backend, se entregarán los archivos y scripts necesarios para realizar el despliegue de la aplicación en la máquina que se desee emplear como servidor, junto con instrucciones para replicar el proceso.

3.3. Calendario del proyecto, división del trabajo, coordinación, comunicaciones, monitorización y seguimiento

A continuación, se presentan la planificación temporal y la organización interna que han permitido la gestión eficiente del proyecto. Esto abarca tanto la división equitativa de las tareas entre los equipos como los mecanismos establecidos para asegurar una comunicación fluida, una coordinación efectiva y una monitorización continua del progreso alcanzado.

En este proyecto estamos trabajando 8 personas, divididos en 3 grupos. En el frontend de escritorio están trabajando Javier Salafranca, José Manuel Vivas y Diego Viñuales; en el frontend de móvil están trabajando Javier Domingo, Alberto Serrano y Óscar Ventas y en backend están trabajando Jorge Soria y Alejandro Villacampa.



Tareas	Grupo	Inicio	Fin
Fase Inicial de Planificación del Proyecto	TODOS	2/2/2025	11/2/2025
Interfaz Inicio Sesión y Registro	frontend-...	14/2/2025	28/2/2025
Conectar Inicio de Sesion	frontend-...	28/2/2025	16/3/2025
Interfaz Menu Principal	frontend-...	28/2/2025	2/3/2025
Conectar Menu Principal	frontend-...	2/3/2025	12/3/2025
Añadir Pantalla Settings	frontend-...	4/3/2025	5/3/2025
Añadir Sistema de Amigos	frontend-...	7/3/2025	9/3/2025
Añadir Pantalla Perfil y Estadisticas	frontend-...	7/3/2025	9/3/2025
Añadir Inventario	frontend-...	7/3/2025	10/3/2025
Añadir Pantalla Juego Basica	frontend-...	12/3/2025	14/3/2025
Interfaz Pantalla de Juego 1v1	frontend-...	17/3/2025	24/3/2025
Conectar Pantalla de Juego 1v1	frontend-...	24/3/2025	31/3/2025
Interfaz Pantalla de Juego 2v2	frontend-...	24/3/2025	31/3/2025
Conectar Pantalla de Juego 2v2	frontend-...	31/3/2025	7/4/2025
Conectar Partidas Amistosas	frontend-...	7/4/2025	14/4/2025
Conectar Partidas Equipo 2v2	frontend-...	7/4/2025	14/4/2025
Añadir Guardado de Partida Amistosa	frontend-...	7/4/2025	14/4/2025
Añadir Rankings 1v1 y 2v2	frontend-...	14/4/2025	21/4/2025
Añadir Tapetes y Barajas seleccionables	frontend-...	14/4/2025	21/4/2025
Añadir Niveles/Rangos	frontend-...	14/4/2025	21/4/2025
Implementación de widgets utiles	frontend-...	28/2/2025	13/3/2025
Interfaz Pantalla de Welcome	frontend-...	28/2/2025	28/2/2025
Interfaz Pantalla de Registro	frontend-...	5/3/2025	5/3/2025
Interfaz Pantalla de Login	frontend-...	1/3/2025	1/3/2025
Interfaz Pantalla de Perfil	frontend-...	9/3/2025	10/3/2025
Interfaz Pantalla de menu principal	frontend-...	6/3/2025	13/3/2025
Interfaz Pantallas de Amigos	frontend-...	8/3/2025	9/3/2025
Interfaz Pantalla de Juegos 1v1	frontend-...	12/3/2025	13/3/2025
Conectar Welcome	frontend-...	2/3/2025	4/3/2025
Conectar Login	frontend-...	4/3/2025	5/3/2025
Conectar Register	frontend-...	4/3/2025	5/3/2025
Conectar Perfil	frontend-...	11/3/2025	11/3/2025
Conectar Estadisticas	frontend-...	11/3/2025	11/3/2025
Conectar menu principal	frontend-...	13/3/2025	13/3/2025
Conectar Amigos	frontend-...	9/3/2025	9/3/2025
Tests inicio	frontend-...	1/3/2025	1/3/2025
Tests Login	frontend-...	5/3/2025	6/3/2025
Tests Register	frontend-...	5/3/2025	6/3/2025
Tests Perfil	frontend-...	12/3/2025	12/3/2025
Interfaz Pantalla de Juego 1v1	frontend-...	17/3/2025	24/3/2025
Conectar Pantalla de Juego 1v1	frontend-...	24/3/2025	31/3/2025
Interfaz Pantalla de Juego 2v2	frontend-...	24/3/2025	31/3/2025
Conectar Pantalla de Juego 2v2	frontend-...	31/3/2025	7/4/2025
Conectar Partidas Amistosas	frontend-...	7/4/2025	14/4/2025
Conectar Partidas Equipo 2v2	frontend-...	7/4/2025	14/4/2025
Añadir Guardado de Partida Amistosa	frontend-...	7/4/2025	14/4/2025
Añadir Rankings 1v1 y 2v2	frontend-...	14/4/2025	21/4/2025
Añadir Tapetes y Barajas seleccionables	frontend-...	14/4/2025	21/4/2025
Añadir Niveles/Rangos	frontend-...	14/4/2025	21/4/2025
Estructura base del proyecto	backend	26/2/2025	26/2/2025
Inicio de sesión, crear cuentas y eliminar cuentas	backend	26/2/2025	28/2/2025
Gestión amigos (lista de amigos, solicitudes...)	backend	28/3/2025	8/3/2025
Docker y configuración	backend	1/3/2025	1/3/2025
Pruebas/tests gestión de usuario y de amigos	backend	1/3/2025	4/3/2025
Estadisticas y primer acercamiento a partidas	backend	2/5/2025	11/3/2025
Chats entre amigos	backend	10/3/2025	12/3/2025
Chats en partidas	backend	10/3/2025	30/3/2025
Partidas	backend	10/3/2025	7/4/2025
Paginación y optimizaciones	backend	5/4/2025	14/4/2025
Ultimas correcciones	backend	7/4/2025	14/4/2025
Finalización de Proyecto y Documentación	TODOS	14/4/2025	26/4/2025

Figura 3.3.1: Diagrama de Gantt con las tareas a realizar.

En los tres grupos de trabajo este ha sido repartido de manera uniforme entre los parti-

cipantes, con todos los integrantes del grupo realizando tareas de código, documentación y diseño. Debido a la naturaleza del frontend, la mayoría de la labor de diseño gráfico ha sido llevada a cabo por los integrantes de estos grupos, empleando un canal de *Discord* para compartir recursos gráficos entre los dos grupos, escritorio y móvil. De manera análoga, gran parte del diseño técnico de la aplicación ha sido llevado a cabo por el grupo de backend, que ha usado el sistema de *issues* de *GitHub* para aceptar y gestionar las peticiones de los grupos de frontend.

La mayoría de la comunicación interna entre los grupos y sus integrantes se ha llevado a cabo a través de un servidor de *Discord*, que contiene una serie de canales de voz y texto con funciones específicas para facilitar la organización de las comunicaciones.

Actualmente existen 3 canales privados, uno para cada grupo de trabajo, usados para comunicarse con los otros integrantes de tu grupo; un canal general, que ha sido usado para mandar mensajes a todos los integrantes o pedir opiniones sobre decisiones de diseño estético; un canal de actas, por el que se manda al final de cada reunión un resumen de lo ocurrido además de las decisiones tomadas; tres canales, **relaciones-públicas**, **recursos** y **avisos**, usados para informar sobre e-mails del tutor, información útil externa al proyecto o documentación sobre el uso de recursos internos; un canal documentos para adjuntar documentos o recursos gráficos importantes ;y un canal registrar-tiempo, que contiene un bot que usamos para guardar en una tabla el tiempo trabajado de cada integrante.

Además del servidor de *Discord*, tenemos también un canal de *Whatsapp* que no ve mucho uso y el repositorio de backend tiene un tag *External request* para las *issues* que permite solicitar funcionalidades del backend a los que trabajan en el frontend.

A la hora de organizar lo que hace cada integrante de cada grupo, hemos creado tres repositorios de *GitHub* con *issues* independientes para que cada grupo cree y asigne *issues* y *sub-issues* de manera interna a cada integrante del grupo. De esta forma, los miembros de un grupo en concreto solo tienen que preocuparse por la división de trabajo de su grupo y no por la del resto. Al inicio del proyecto, creamos en cada repositorio una serie de *issues* bastante generales a las que, a medida que avanzábamos en el proyecto, les fuimos añadiendo *sub-issues* más específicas asignadas a distintos miembros en específico del grupo, especificando así qué parte hace cada miembro.

La manera que hemos empleado de monitorizar el progreso de cada grupo, y del proyecto en general, es ver cuántos requisitos funcionales están completados y qué porcentaje de las *issues* generales están ya cerradas. Como creamos bastantes *issues*, tanto para solucionar bugs como para especificar partes o funcionalidades que completar, mirar el porcentaje total de *issues* completadas no da mucha información y es por ello que usamos más los requisitos funcionales completados y las *issues* generales cerradas como métricas de progreso.

Otra métrica que usamos son las horas de trabajo de cada integrante, ya que las tablas de horas dedicadas son públicas, aunque una alta cantidad de horas no significa necesariamente progreso en el proyecto. Las horas dedicadas se miran al final de cada semana mientras que las *issues* generales cerradas las mira cada grupo a medida que avanza en el proyecto.

En caso de que alguien no trabajara lo suficiente o en caso de que hubiera cualquier problema entre dos integrantes del grupo, hemos elegido a un director, Javier Salafranca, que se encargaría de hablar con ellos para intentar solucionar el problema. Además, cada grupo siempre tiene la opción de hablar con el integrante problemático de manera directa sin recurrir al director. En caso de que hubiera problemas con el horario y viéramos que un grupo no avanza lo suficiente, también tenemos la opción de reasignar a gente de otros grupos a este grupo para ayudar con la carga de trabajo.

4. Análisis y diseño del sistema

4.1. Análisis de requisitos

A continuación, se detallan los requisitos de la aplicación en desarrollo. Para facilitar su visualización, se presentan en forma de tabla, en la que cada fila representará un requisito concreto:

ID	**Descripción**	**Forma de verificarlo**
RF1	El usuario debe poder crear una cuenta en la aplicación utilizando un correo electrónico, un nombre de usuario (único) y contraseña introducidos por él.	Podemos verificar que al completar el proceso de creación de cuenta dentro de la aplicación, el resultado se refleja en la base de datos de usuarios.
RF1.1	El usuario debe poder emplear el nombre de usuario y la contraseña de una cuenta previamente creada para iniciar sesión y cargar los datos asociados a esta.	Se puede comprobar que, tras haber creado una cuenta con unos parámetros determinados, si posteriormente se inicia sesión con dicha cuenta, los parámetros son los correspondientes.
RF1.2	Debería ser posible para el usuario cerrar sesión, es decir, descargar toda la información asociada a su cuenta del dispositivo en el que está y poder iniciar sesión con otra cuenta distinta.	Validar que al seleccionar la opción de cerrar sesión, la aplicación permite iniciar sesión con otra cuenta y que los datos cargados de esta sean los propios y no los de la anterior.
RF2	El juego permitirá a los jugadores buscar e iniciar partidas de guiñote entre dos jugadores, uno contra uno, con límite de tiempo por turnos y las reglas estándar del guiñote, emparejados por un sistema de puntos de habilidad.	La verificación del requisito se puede hacer buscando una partida usando la interfaz de la aplicación y comprobando que la partida se desenvuelve correctamente, así como constatar que el emparejamiento por puntos de habilidad es preciso.
RF2.1	Además de partidas uno contra uno, debe existir la opción de buscar partidas por parejas, es decir: dos jugadores enfrentándose a otros dos jugadores, siendo cada una de las parejas formada por usuarios considerados “amigos” por el juego.	Conforme al requisito previo, se puede verificar la corrección de este buscando una partida por parejas usando la interfaz de la aplicación y comprobando que el juego se desenvuelve de forma correcta.

RF2.2	Se incluirá un sistema de puntos de habilidad que funcione de la siguiente manera: cuando un usuario gana una partida, dependiendo del número de puntos que ya posea, obtendrá un número determinado extra de puntos.	Habrá que comprobar que la función que calcula el número de puntos que se obtienen al vencer y al perder en una partida se calcula correctamente en la aplicación.
RF3	Se incluirá la opción de jugar partidas de forma privada con usuarios considerados "amigos", a los cuales el usuario podrá solicitar el juego de una partida.	Se comprobará que, si un usuario pide el comienzo de una partida a un amigo y este acepta la petición, el juego comienza correctamente.
RF3.1	Se permitirá al jugador, únicamente dentro de las partidas contra amigos, modificar algunas reglas para hacerlas más únicas y diferentes a las habituales.	Este requisito puede evaluarse iniciando una partida personalizada con unas determinadas reglas contra un usuario amigo y validando que las reglas seleccionadas afectan como se espera al juego.
RF4	Se implementará un *sistema de pausa* que permitirá detener una partida cuando *todos* los jugadores participantes (en partidas de 1 vs 1 o 2 vs 2) soliciten la pausa. Al activarse, deberán conservarse sin alteración las cartas en mano, el mazo restante, el turno activo, el tiempo restante de dicho turno y la puntuación acumulada. La partida permanecerá congelada hasta que todos los jugadores indiquen su deseo de reanudarla.	Para verificar este requisito se iniciará una partida, se avanzará hasta un estado intermedio y, a continuación, cada jugador enviará la solicitud de pausa. Se comprobará que: (i) no se permiten nuevas acciones mientras la partida esté pausada; (ii) el temporizador se detiene y conserva su valor; (iii) tras la reanudación, las cartas, los puntos y el turno continúan exactamente como antes de la pausa.
RF5	Dentro de una partida, tanto individual como por parejas, se permitirá al jugador enviar y recibir mensajes en un formato de chat de texto.	Se verifica el envío correcto de un mensaje y su posterior recepción, garantizando la integridad y que lo ha recibido el usuario correcto.
RF6	La aplicación tendrá una funcionalidad de lista de amigos que permitirá enviar y recibir peticiones de amistad.	Se debe corroborar que los usuarios pueden buscar a otros por nombre, enviar peticiones de amistad y que la acción de aceptar o rechazar la solicitud se refleja correctamente.
RF6.1	Además de agregar amigos, el usuario debería poder eliminarlos de la lista de forma automática sin requerir confirmación del otro usuario.	Se verificará que, si un usuario elimina a otro de su lista de amigos a través de la interfaz, esto se refleja correctamente en la base de datos y ambos se eliminan mutuamente.

RF7	Se podrá personalizar tanto las barajas como los tapetes que se ven en partida, y los ítems de personalización se obtienen a medida que se asciende en rangos.	Se ratifica que, cuando un jugador selecciona una baraja o tapete, la presentación visual de la partida cambia conforme a la elección del jugador.
RF8	A partir de los puntos de habilidad, el progreso en el juego se fracciona en rangos, cada uno con un título y posibles recompensas.	Se deberá comprobar que, cuando un jugador alcanza el número de puntos requerido para un nuevo rango, se modifican los parámetros específicos para dicho jugador.
RF9	Existirá un "ranking" con una lista ordenada por puntos de habilidad de todos los jugadores registrados en la aplicación.	Se verificará que los rankings mostrados al jugador son válidos y que el ranking privado solo muestra a los amigos del usuario, mientras que el global incluye a todos los jugadores.
RF10	Un usuario podrá visualizar las estadísticas de los amigos agregados en su perfil, incluyendo partidas ganadas, perdidas y racha de victorias.	Se comprobará que las estadísticas mostradas de un usuario son las mismas para toda la base de jugadores en un momento dado y que no muestran información de otro usuario.

En este apartado, se especifican las reglas disponibles para modificar en las partidas personalizadas privadas:

- Es posible alterar el formato, es decir: el número de rondas necesarias para ganar: puede ser al mejor de 5, al mejor de 3, etc.
- También se pueden cambiar los límites de tiempo por turno, para poder configurar sesiones más ágiles o más pausadas, dependiendo de las especificaciones.
- Finalmente, se podrá ajustar la puntuación necesaria para vencer en una ronda.

4.2. Diseño del sistema

En esta sección se describe el diseño del sistema siguiendo un estilo arquitectónico de **capas** que enfatiza la claridad y separación de responsabilidades. La arquitectura principal del sistema es una **arquitectura de tres niveles (capa de presentación, capa de aplicación y capa de datos)**.

A lo largo de los siguientes apartados se verá cómo este enfoque de capas se materializa en diagramas de módulos para backend y frontends, cómo se traduce en una vista de componentes y conectores, y cómo se plasma finalmente en el diagrama de despliegue con contenedores Docker y nodos físicos/virtuales. Cada diagrama irá acompañado de su justificación, explicando el motivo por el cual se ha elegido cada patrón de diseño y cómo contribuye a la escalabilidad y a la evolución independiente de cada subsistema.

4.2.1. Diagrama de módulos Backend

A continuación se muestra un diagrama que refleja la división modular del sistema backend. Cada módulo agrupa funcionalidades relacionadas, lo que facilita la organización del código, el trabajo en equipo y el cumplimiento de los distintos requisitos del proyecto. Además, se explicará el patrón de acceso a la base de datos, se justificará el criterio de división de módulos, etc.

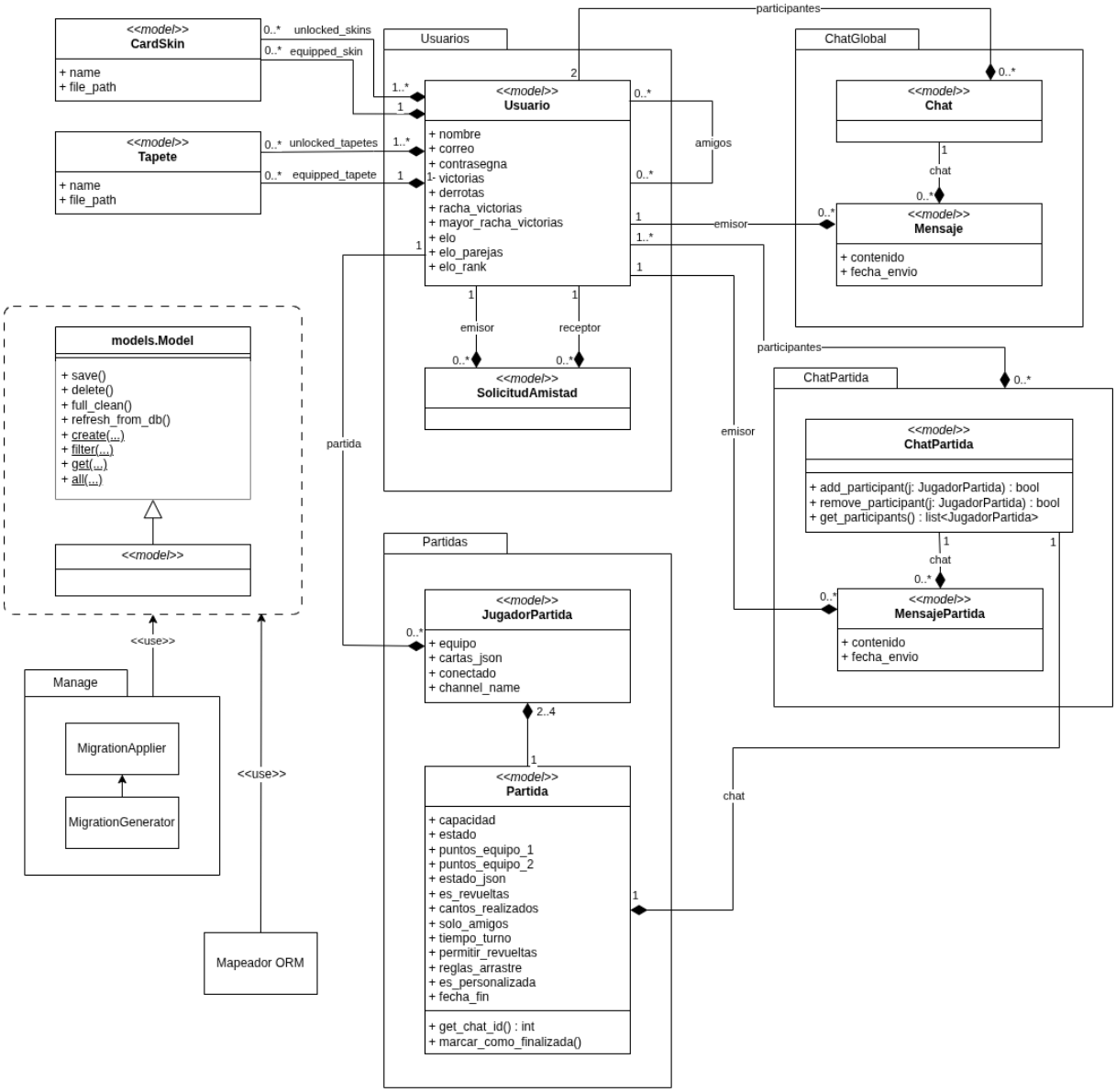


Figura 4.2.1: Diagrama UML de los módulos del backend.

Patrón de acceso a la base de datos

En Django, el acceso a datos se basa en el patrón de diseño **Active Record**, que permite la interacción con bases de datos a través de la programación orientada a objetos.

Las clases identificadas por el estereotipo «model» heredan de models.Model y representan entidades del dominio. Para mantener la claridad del diagrama, esta relación de generalización / herencia se ha representado una única vez mediante una clase genérica en el área delimitada por un rectángulo punteado.

Cada clase que hereda de models.Model se corresponde con una tabla en la base de datos, mientras que cada instancia de la clase representa un registro o fila. Estas clases no solo definen la estructura de los datos (campos y atributos), sino que también heredan de su clase base una serie de métodos (p.e. save, delete, create, filter...) que permiten realizar operaciones de creación, consulta, modificación y eliminación de registros, usando un sistema de mapeo objeto-relacional (ORM) integrado en Django.

El archivo manage.py permite ejecutar comandos como makemigrations o migrate que traducen estas clases modelo a sentencias SQL compatibles con el sistema gestor de bases de datos usado, en este caso PostgreSQL. En el diagrama, estas funcionalidades están representadas mediante los módulos MigrationGenerator y MigrationApplier, que interactúan con las clases modelo para generar y aplicar migraciones.

Aunque los modelos encapsulan la lógica de persistencia, no cumplen el rol de controladores. En el esquema de arquitectura de Django, esta responsabilidad recae sobre las views (no representadas en el diagrama). Las vistas gestionan las solicitudes del usuario, interactúan con los modelos y devuelven la respuesta correspondiente.

Criterio para la división de módulos

Se ha decidido emplear el patrón de diseño **Active Record** junto con el mapeador objeto-relacional **(ORM)** de Django, ya que permite reducir el acoplamiento entre componentes (loose coupling), escribir menos código y aplicar el **principio DRY** (Don't Repeat Yourself), evitando la necesidad de definir manualmente operaciones de lectura y escritura para cada tabla.

Para estructurar la aplicación, se ha optado por dividir el sistema en varios módulos principales como Usuarios, Partidas y ChatGlobal. Aunque están relacionados entre sí, presentan diferencias funcionales suficientes para justificar su separación. Esta división facilita la distribución de tareas entre los integrantes del equipo. Además, dichos módulos se centran más en distintos requisitos del sistema. Por ejemplo, el módulo de Usuarios gestiona el registro y la autenticación, Partidas se encarga del emparejamiento y desarrollo de partidas de guiñote, y ChatGlobal ofrece funcionalidades de mensajería entre usuarios. Esto no significa que los módulos funcionen de forma completamente aislada, ya que en muchos casos es necesaria la colaboración entre ellos para completar o precisar ciertos requisitos del sistema. Por ejemplo, el módulo de Usuarios es esencial para identificar a los jugadores durante el emparejamiento en Partidas.

Cada módulo cuenta con sus propias views, que hacen de controladores y ofrecen las funcionalidades a través de una API REST. En algunos casos, también se usan channels para manejar comunicación en tiempo real, como en el chat o durante las partidas.

Bibliotecas utilizadas en los módulos y justificación de uso

A continuación se mencionan las bibliotecas externas utilizadas en los módulos y se

justifica su uso:

- **Django:** Framework principal del desarrollo del backend, más adelante se explicará en detalle la decisión de este framework.
- **Django Rest Framework (DRF)**: Para la creación de APIs RESTful. Proporciona herramientas que facilitan la creación APIs robustas y fiables. Utilizada por integración directa con Django.
- **PyJWT**: Utilizada para la gestión de JSON Web Tokens para la autenticación. Utilizado nuevamente por integración con Django.
- **Daphne:** Servidor ASGI para manejar conexiones WebSocket. Necesario para la comunicación en tiempo real y elegido frente a otras por su integración con Django Channels.
- **Django Channels:** Implementación de WebSockets y canales en Django. Necesario para la conexión en tiempo real.
- **Psycopg2:** Conector de PostgreSQL para Python. Utilizada para conectar e interactuar con la base de datos en PostgreSQL.

4.2.2. Diagrama de módulos frontend móvil

En esta sección se presenta el diagrama de módulos del frontend móvil, donde se identifican los paquetes principales de la aplicación Flutter y sus relaciones, reflejando la separación de responsabilidades entre UI, lógica de aplicación y acceso a datos.

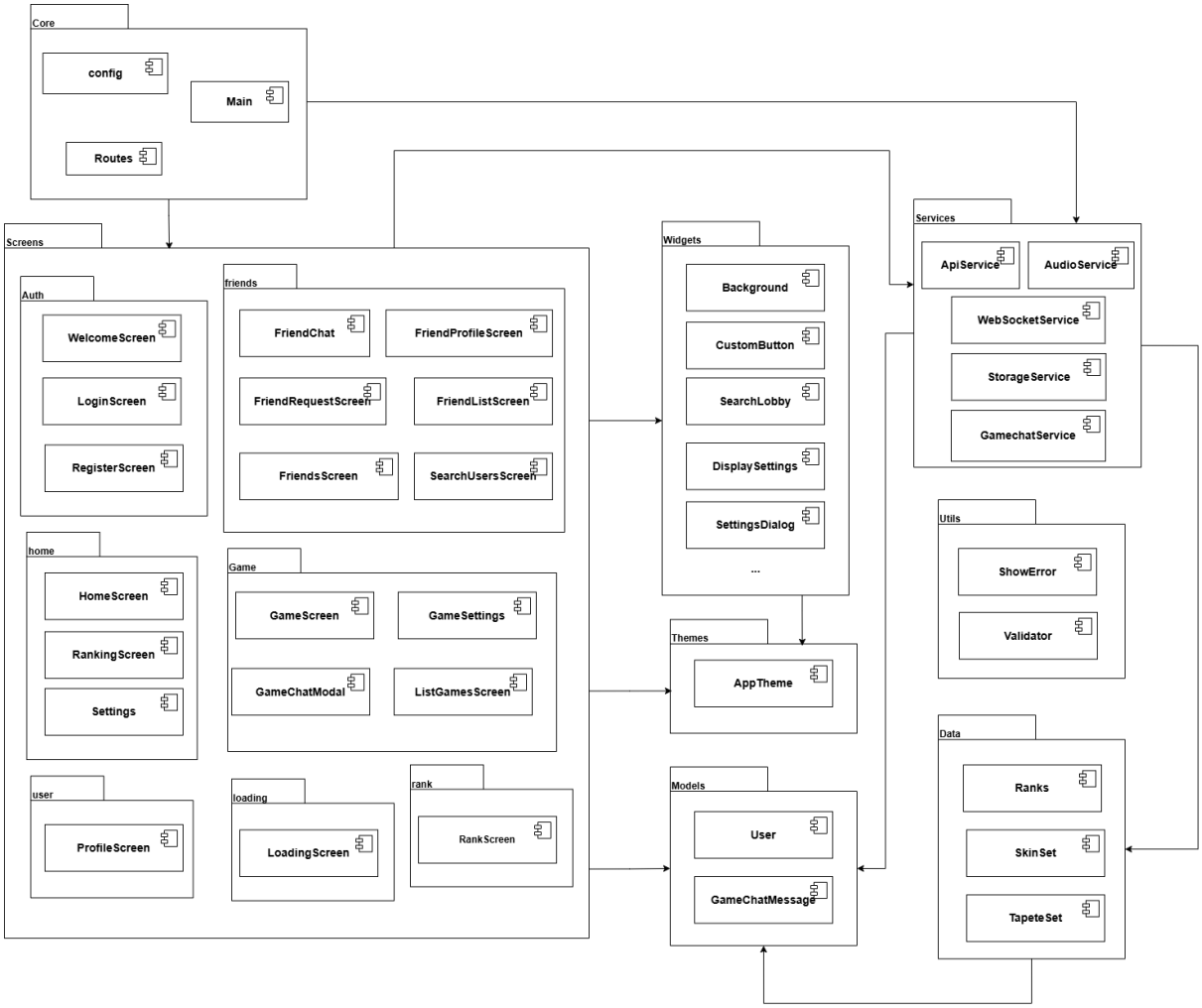


Figura 4.2.2: Diagrama UML de los módulos del frontend de móvil.

Paquete Core (Coordinator + Singleton): Agrupa `main.dart`, `config.dart` y `routes.dart`. Actúa como núcleo de arranque: lee constantes (endpoints, timeouts), registra proveedores (Provider) y mapea rutas a los paquetes de pantalla. Este único punto de entrada evita replicar lógica de inicialización y navegación.

Paquete Screens (Composite + Dependency Injection + Observer): Este paquete agrupa las pantallas de la aplicación en submódulos por dominio, construidos como un árbol de widgets (Composite), con servicios inyectados vía Provider (Dependency Injection) y su suscripción a flujos de datos en tiempo real (Observer).

Todos los paquetes listados a continuación usan el **módulo de Widgets** para reciclar elementos de la UI y el módulo Theme, para mantener el mismo estilo en la aplicación

y evitar la duplicidad innecesaria de código. Además, varios módulos emplean utilidades del módulo **Utils**.

- **Screens/auth**: Agrupa `WelcomeScreen`, `LoginScreen` y `RegisterScreen`. Comparten ApiService y StorageService inyectados para gestionar el flujo de autenticación (carga, validación y almacenamiento de credenciales) sin dispersar lógica en cada widget, delegando la navegación al núcleo de rutas.
- **Screens/home**: Agrupa `HomeScreen`, `RankingScreen` y `Settings` consumen la misma instancia de ApiService (Repository) y StorageService (Singleton), garantizando la consistencia de datos en todas las pantallas y simplificando el acceso a las estadísticas y a las preferencias.
- **Screens/game**: `ListGamesScreen`, `GameScreen` y `GameChatModal` se suscriben a WebSocketService (Observer) para gestionar las partidas y el chat de las propias partidas en tiempor real, manteniendo así a los widgets libres de lógica de red.
- **Screens/friends**: `FriendListScreen`, `FriendRequestScreen`, `FriendChatScreen`, etc usan ApiService para CRUD de contactos vía **REST**.
- **Screens/user**: Carga las skins y los tapetes del usuario desde el módulo **data** y muestra el perfil con los servicios de ApiService y StorageService.
- **Screens/loading**: Reproduce un vídeo inicial usando VideoPlayer, y tras un temporizador (Procedural Controller), redirige a la pantalla de bienvenida, aislando el flujo de splash en un único widget.
- **Screens/rank**: Obtiene los datos de rango y elo mediante ApiService y muestra el progreso con RankProgressBar de Widgets (Composite + DI).

Paquete Widgets (Composite + Strategy): Se implementan varios componentes genéricos y reutilizables con parámetros de estilo y callbacks, aplicando Strategy para intercambiar temas o comportamientos sin tocar su lógica interna.

Paquete Services (Repository + Singleton): Centraliza los servicios utilizados en toda la aplicación. Como Repository abstrae llamadas REST/Websocket y manejo de errores; como Singleton unifica la configuración de clientes.

Paquetes Data (Adapter) y Models (Factory Method): Adaptan respuestas de services al dominio (User, GameChatMessage), cuyos métodos `fromJson()` y `toJson()` aíslan la estructura de la API. facilitando cambios de contrato sin alterar la presentación.

Paquetes Utils (Helper) y Theme (Strategy): Centralizan validaciones, diálogos de error, paletas de colores y estilos tipográficos.

Dependencias y bibliotecas externas usadas:

Para soportar el diseño descrito, son necesarias las siguientes dependencias:

- **flutter/material**: Proporciona widgets y estilos base.

- **web_socket_channel (2.4.0)**: Proporciona canales WebSocket en tiempo real.
- **http (1.3.0)**: Para las peticiones REST y el manejo de respuestas asíncronas.
- **carousel_slider (5.0.0)**: Para crear carruseles de imágenes y contenido desplazable en la UI.
- **image_picker (1.0.0)**: Proporciona acceso a la galería y cámara para selección de imágenes de usuario.
- **video_player (2.5.5)**: Reproduce vídeo en la pantalla de carga.
- **smooth_page_indicator(1.2.1)**: Añade indicadores animados de página para carruseles.
- **audioplayers (6.4.0)**: Reproduce efectos de sonido y audio de forma simple.
- **shared_preferences (2.5.2)**: Guarda preferencias ligeras de manera persistente.
- **flutter_secure_storage (9.2.4)**: Almacena tokens y credenciales de forma cifrada y segura.
- **intl (0.17.0)**: Formatea fechas, horas y textos según la localización del usuario.

4.2.3. Diagrama de módulos Frontend escritorio

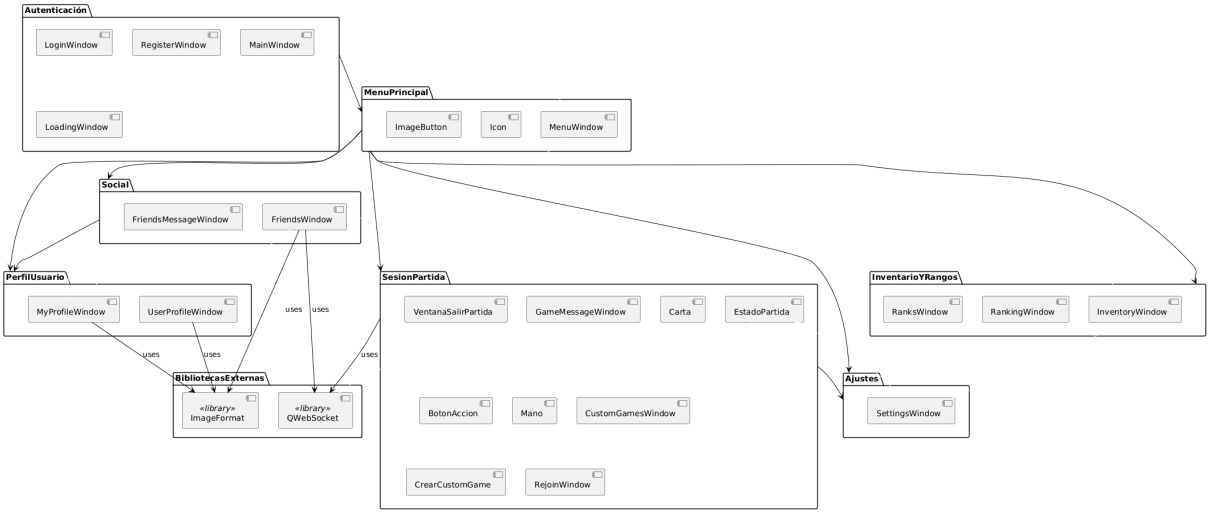


Figura 4.2.3: Diagrama UML de los módulos del frontend de escritorio.

En este diagrama podemos ver los distintos módulos y clases usados a lo largo del proyecto, con cada módulo representando una pantalla distinta.

El módulo **Autenticación** representa la pantalla inicial de la aplicación cuya función principal es permitir al usuario iniciar sesión y crear una cuenta. Para hacer esto, usa clases como LoginWindow y RegisterWindow. Además, también usa la clase LoadingWindow para la transición al menú principal, representado por el módulo **MenuPrincipal**.

El módulo **MenuPrincipal** sirve como el menú principal de la aplicación desde el cual puedes acceder al resto de funcionalidades como el inventario, sistema de amigos o partidas. Usa principalmente las clases Icon e ImageButton para representar elementos de la UI como los botones para ir a otros menús o los botones para jugar partidas. Desde este módulo podemos acceder a los módulos **InventarioYRangos**, **Social, PerfilUsuario, SesionPartida y Ajustes**.

El módulo **InventarioYRangos** representa tres menús accesibles desde **MenuPrincipal**: el menú de inventario, que te permite gestionar tus skins y tapetes; el menú de rangos, que te permite ver qué rango eres y cuánto te queda para llegar al siguiente rango; y el menú de rankings. que te muestra la posición global de cada jugador según sus puntos. Cada uno de estos menús se representa mediante una clase distinta, **RanksWindow**, **RankingWindow** e **InventoryWindow**.

El módulo **Social** representa el sistema de gestión de amigos de la aplicación y para ello contiene una clase FriendsWindow que se encarga del listado, búsqueda y solicitudes de amistad; y una clase FriendsMessageWindow que gestiona los mensajes privados entre amigos. Para representar el perfil de tus amigos, el módulo **Social** usa el módulo **PerfilUsuario**.

El módulo **PerfilUsuario** es el encargado de representar tanto tu perfil como el de

otros jugadores, además de las estadísticas asociadas a tu cuenta. Para ello, emplea dos clases distintas: MyProfileWindow y UserProfileWindow.

El módulo **SesionPartida** es el encargado de representar las partidas jugadas entre dos o cuatro jugadores distintos en la aplicación. Además de gestionar las partidas como tal, también se encarga de tratar la reconexión a partidas, la creación de partidas personalizadas y el sistema de mensajes entre jugadores en la misma partida.

Para representar la partida se usan una serie de clases como Carta, Mano, EstadoPartida, BotonAcción o VentanaSalirPartida. El sistema de mensajería se encuentra en la clase GameMessageWindow y la reconexión de partidas en ReconectWindow. La creación de partidas personalizables y la conexión a ellas usa las clases CustomGameWindow y CrearCustomGame.

El módulo **Ajustes** representa el menú que permite cambiar el volumen de la música y efectos de sonido y es usado tanto en **MenuPrincipal** como **SesionPartida**, el módulo entero se representa por una sola clase, SettingsWindow.

El módulo de **BibliotecasExternas** representa las bibliotecas usadas externas a la aplicación, entre ellas cabe destacar las siguientes:

- QWebSocket: Para la conexión con los sockets usados en el backend para el sistema de mensajería entre jugadores y el sistema de comunicación con backend necesario para jugar partidas.
- QImageFormats: Para poder interpretar imágenes de formato .webp a la hora de gestionar imágenes de perfil.

Diagrama de componentes y conectores.

- **Diagrama UML:**

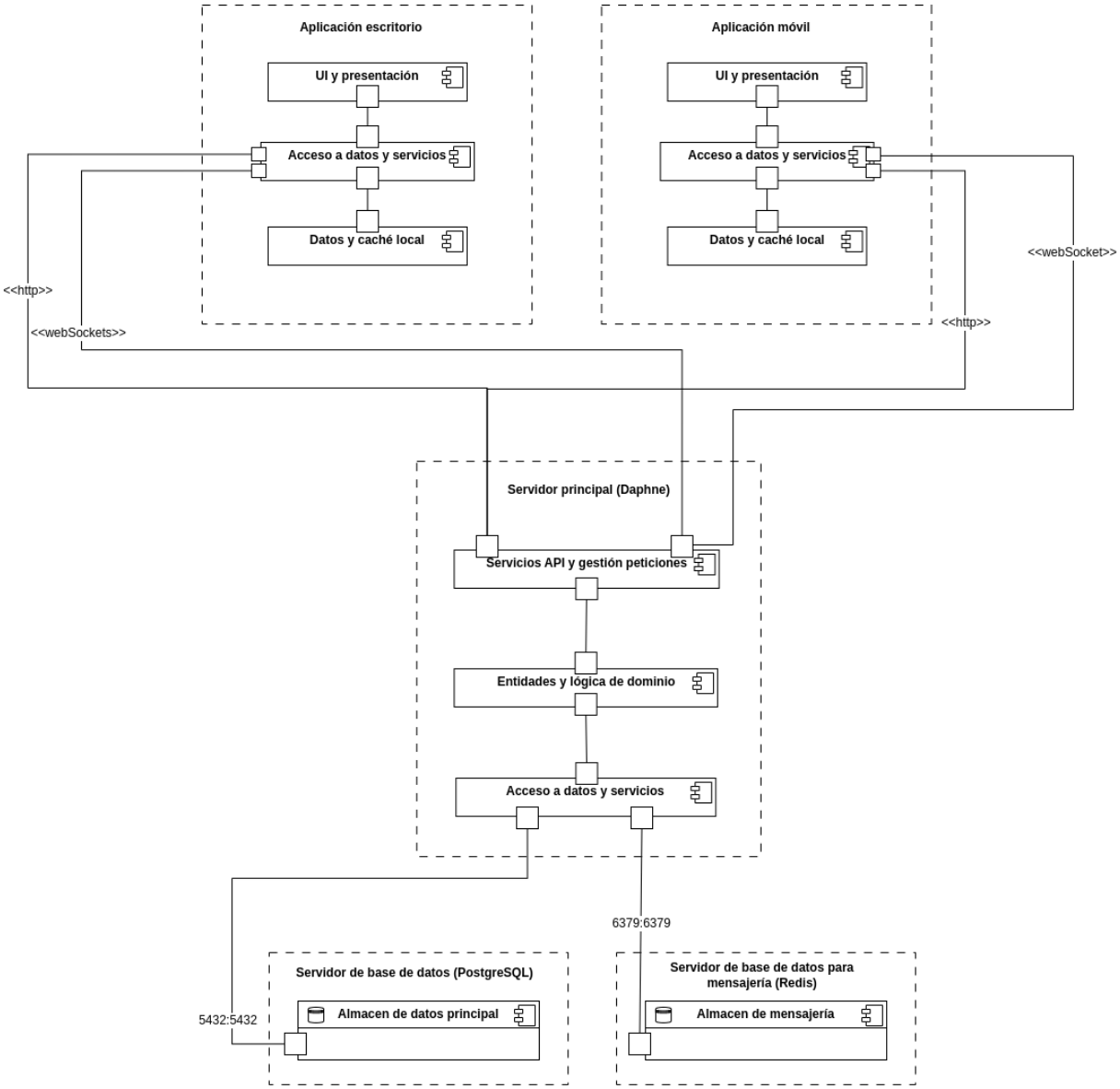


Figura 4.2.4: Diagrama UML de componentes y conectores.

- **Elementos, relaciones y propiedades:** El sistema se compone de varios elementos interconectados para garantizar la gestión eficiente de datos y servicios. En primer lugar, el servidor de la base de datos principal, **PostgreSQL**, operando en el puerto **5432**, se conecta al servidor principal **Daphne** a través del módulo de acceso a datos y servicios. De manera similar, el servidor de base de datos para mensajería, **Redis**, operando en el puerto **6379**, también se conecta a **Daphne** a través de este mismo módulo.

  El módulo de acceso a datos y servicios gestiona la comunicación con las entidades y la lógica del dominio, las cuales se encargan del procesamiento central del sistema.

Estas entidades, a su vez, se conectan a los servicios **API** y al sistema de gestión de peticiones, que facilitan la interacción con las aplicaciones cliente.

Las aplicaciones, tanto móvil como de escritorio, acceden al sistema mediante el protocolo **HTTP**, conectándose a sus respectivos módulos de acceso a datos y servicios. Estos módulos, además, están integrados con la interfaz de usuario y presentación de cada aplicación, así como con sus respectivos sistemas de almacenamiento local y caché. Adicionalmente, la comunicación en tiempo real entre las aplicaciones y el backend se establece mediante **WebSockets**, facilitando una interacción fluida y eficiente.

- **Interfaces y comportamiento:**

  Se ha creado un **API web RESTful** para gestionar funcionalidades básicas como la gestión de usuarios, la gestión de solicitudes de amistad, de estadísticas, etc.

  Las direcciones de los endpoints vienen determinadas por un prefijo que indica el conjunto de conocimiento que trata:

Prefijo	**Funcionalidades**
`/usuarios/{...}`	Creación de usuarios, inicio de sesión o eliminación de cuentas, obtener listado de amigos, etc.
	Obtener estadísticas y otras métricas (racha, porcentaje de victorias...), obtener rankings, etc.
`/amigos/{...}`	Enviar solicitudes de amistad, aceptar o denegar una solicitud, listar solicitudes de amistad, buscar usuarios por nombre, etc.
`/mensajes/{...}`	Enviar mensajes o obtener mensajes en chats individuales con un amigo.
`/partida/{...}`	Crear partidas, gestionar el estado de una partida, obtener su estado, etc.
`/chat/{...}`	Gestionar chat dentro de una partida.

Tabla 2: Tabla de prefijos y funcionalidades en la API

Los endpoints soportan métodos HTTP como GET, DELETE o POST.

Se ha usado autenticación con **JSON Web Tokens**. Cuando un usuario crea una cuenta o inicia sesión mediante la petición correspondiente recibirá un token. El Frontend deberá almacenarlo convenientemente. Algunas peticiones requieren poner el token en la cabecera de la petición (para autenticar al usuario) o poner parámetros específicos en el cuerpo de la petición o como parte de la URL.

Para consultar los endpoints específicos, sus especificaciones, los parámetros que requieren, mensajes de error y el formato de respuesta... se podrá consultar en la *Wiki* del repositorio del Backend.

Además para funcionalidades que requieren de comunicación bidireccional cómo obtener actualizaciones del chat en tiempo global o actualizaciones en partidas se usará WebSockets, también especificado en la Wiki.

- **Razones:**

  En la exposición de razones se explicará el criterio de elección de REST API como protocolo principal y de WebSockets con Redis para comunicación bidireccional en tiempo real. También se explicará el criterio para elegir JSON Web Tokens como método de autenticación,

  Se ha elegido **REST para la API** por su simplicidad, compatibilidad y facilidad de uso. Además es fácil integrarlo con cualquier cliente que soporte HTTP estándar, en nuestro caso móvil y escritorio.

  Se ha empleado **WebSocket**s para funcionalidades como el chat o las partidas (todavía no está completo) porque permite enviar y recibir datos con gran rapidez sin necesidad de realizar múltiples solicitudes HTTP y permitiendo una conexión persistente.

  Django usa por defecto cabeceras CORS y otros mecanismos contra ataques CSRF. Estos mecanismos de autenticación emplean Cookies o datos de sesión, Como nuestros clientes (escritorio y móvil) no usan Cookies como sí lo haría un cliente web, hemos optado por el uso de **JSON Web Tokens**. La autenticación con JWT permite la autenticación sin estado (los clientes no deberán almacenar datos importantes como la contraseña), son eficientes y además son los recomendados por Django para aplicaciones móviles.

4.2.4. Diagrama de despliegue.

En este apartado se muestra cómo se despliega la aplicación, detallando el nodo principal que contiene los servicios de backend en **contenedores de Docker** y los nodos cliente que consumen la API.

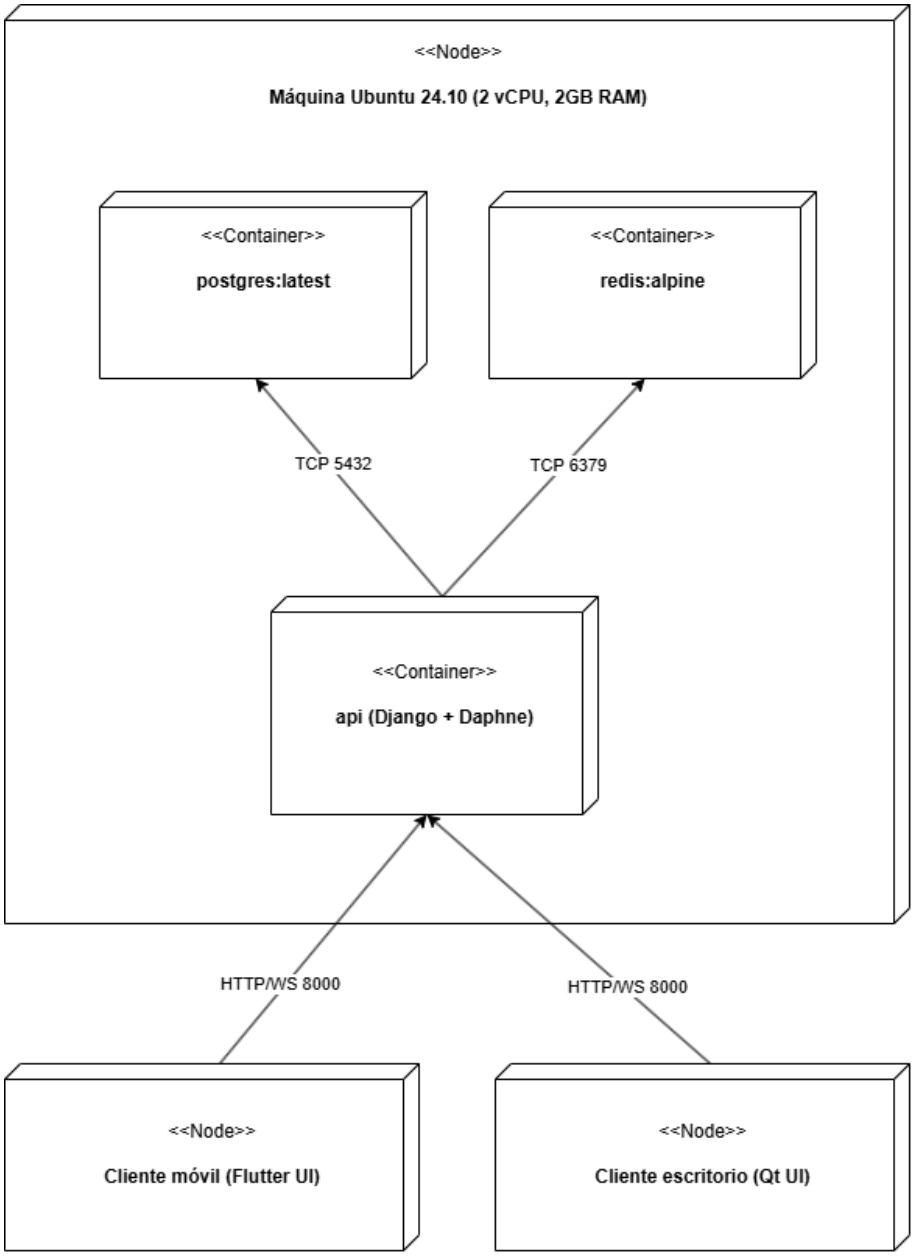


Figura 4.2.5: Diagrama UML del despliegue.

A continuación, se describe cada elemento del diagrama, integrando criterios de diseño y justificación.

- **Máquina Ubuntu 24.10 (2 vCPU, 2GB RAM)**: Nodo físico/virtual que hospeda los servicios backend con Docker, garantizando aislamiento y portabilidad.

- **postgres:latest [TCP 5432]**: Base de datos relacional con transacciones ACID y soporte de réplicas; Docker simplifica la gestión de versiones y la migración de datos.
- **redis:alpine [TCP 6379]**: Almacén en memoria de baja latencia para mensajería en tiempo real.
- **Django + Daphne [HTTP/WS 8000]**: Lógica de negocio y endpoints REST y WebSocket, encapsulada en Docker para entornos idénticos.
- **Cliente móvil** y **Cliente escritorio [HTTP/WS 8000]**: Ambos consumen la API unificada, centralizando rutas y autenticación sin duplicar lógica.

La adopción de Docker en el backend responde a:

1. **Reproducibilidad**: Un único `docker-compose.yml` genera contenedores idénticos en desarrollo, pruebas y producción.
2. **Aislamiento**: Cada servicio corre en su propio contenedor, evitando conflictos de dependencias.
3. **Escalabilidad**: Facilita replicar servicios (p.ej. instancias de API) sin alterar el resto de la infraestructura.

Este despliegue sigue un estilo **cloud-native** basado en una **arquitectura de tres niveles (presentación, aplicación y datos)**, donde cada servicio corre en su propio contenedor Docker para maximizar portabilidad, aislamiento y escalado horizontal.

4.2.5. Mapeo entre vistas.

- **Backend**: Como se observa en la vista de componentes y en la de despliegue, el servidor Daphne ejecuta la aplicación Django, gestionando peticiones HTTP y WebSockets. A través de los puertos 5432 y 6379, se comunica con PostgreSQL para el almacenamiento de datos y con Redis para la mensajería en tiempo real. Además, según el diagrama de módulos Backend, Daphne gestiona las clases del modelo de Django y las transforma en esquemas relacionales que se almacenan en PostgreSQL.
- **Frontend-Escritorio**: El diagrama de despliegue de frontend-escritorio concuerda con los demás ya que solo contiene el app.deb, para correr la aplicación en Linux. Las clases del diagrama de módulos están también representadas en las tres secciones del diagrama de componentes, aunque algunas clases tienen secciones que pertenecen a varias secciones a la vez. Settings, Menu e Inicio pertenecen a la sección de Ui y presentación ya que describen las interfaces de la aplicación. Settings, Menu e Inicio tienen partes de Acceso a datos y servicios ya que se conectan con el backend para extraer datos y acceder a servicios. Tanto Settings como Inicio tienen secciones de Datos y caché local al guardar los datos de configuración y usuario en la máquina cliente.

- **Frontend-Móvil**: El diagrama de despliegue de frontend-móvil concuerda con los demás ya que lo único que podemos ver en él es el app.apk. Las clases representadas en el diagrama de módulos también corresponden a las distintas secciones presentadas en la sección de componentes: Screens, Widgets Themes, Main y Utils pertenecen a Ui y presentación; Services pertenece a Datos y Cache; y Config y Services pertenecen a Acceso a datos.

4.2.6. Mapeo con los requisitos.

Requisito	**Backend**	**Frontend**
RF1	backend → Usuarios, Usuario	escritorio → RegisterWindow móvil → Screens/Auth/RegisterScreen
RF1.1	backend → Usuarios, Usuario	escritorio → LoginWindow móvil → Screens/Auth/LoginScreen
RF1.2	backend → Usuarios, Usuario	escritorio → MyProfileWindow móvil → Screens/User/ProfileScreen
RF2	backend → Partidas, JugadorPartida, Partida	escritorio → MenuWindow móvil → Screens/Home/HomeScreen, Screens/Game/ListGamesScreen
RF2.1	backend → Partidas, Partida	escritorio → CustomGamesWindow móvil → Screens/Game/GameScreen
RF2.2	backend → Partidas, Partida, Usuarios, Usuario	escritorio → UserProfileWindow móvil → Screens/User/ProfileScreen
RF3	backend → Partidas, JugadorPartida, Partida	escritorio → EstadoPartida móvil → Screens/Game/GameScreen
RF3.1	backend → Partidas, Partida	escritorio → CustomGamesWindow móvil → Screens/Game/ListGamesScreen
RF4	backend → Partidas, JugadorPartida, Partida	escritorio → EstadoPartida móvil → Screens/Game/GameSettings
RF5	backend → ChatPartida, ChatGlobal, Mensaje	escritorio → GameMessageWindow movil → Screens/Game/GameChatModal
RF6	backend → Usuario, SolicitudAmistad	escritorio → FriendsWindow móvil → Screens/Friends/FriendListScreen, SearchUsersScreen
RF6.1	backend → Usuario, SolicitudAmistad	escritorio → FriendsWindow móvil → Screens/Friends/FriendsScreen
RF7	backend → Tapete, CardSkin, Usuario	escritorio → InventoryWindow móvil → Screens/User/ProfileScreen

RF8	backend → Usuario	escritorio → RanksWindow móvil → Screens/Rank/RankScreen
RF9	backend → Usuario	escritorio → RankingsWindow móvil → Screens/Home/RankingScreen
RF10	backend → Usuario,SolicitudAmistad	escritorio → UserProfileWindow móvil → Screens/Friends/FriendProfileScreen

4.2.7. Tecnologías elegidas

Lenguajes de programación empleados:

- **Python:** Utilizado para el desarrollo del backend y elegido debido a su facilidad para integrarse con el resto de tecnologías. Se plantearon alternativas como Java, JavaScript o PHP pero la cantidad de herramientas y bibliotecas que ofrece Python nos hizo decantarnos por dicha opción.
- **C++**: Utilizado para el desarrollo del frontend de escritorio y elegido debido a su rendimiento y conocimientos previos por parte de los desarrolladores. Las alternativas propuestas fueron Java o C, pero se descartaron por su complejidad para desarrollar interfaces gráficas.
- **Dart:** Utilizado para el desarrollo del frontend de móvil debido a su capacidad de crear aplicaciones nativas tanto en Android como en iOS. Se planteó usar Java o Kotlin debido a que son el estándar de desarrollo en apps para Android, pero se eligió Dart por la escalabilidad a otras plataformas y sistemas operativos.

Componentes que se integrarán:

- **Django**: Framework de Python utilizado en el backend para crear la API RESTful. Elegido por su robustez y **compatibilidad con el patrón MVC**. Se plantearon otros frameworks como Flask, se descartartó por ser demasiado simple para el tamaño del proyecto.
- **Qt:** Framework de C++ utilizado para el frontend de escritorio. Fue elegido por su rendimiento y capacidad de crear apps nativas. Otras opciones fueron Electron y JavaFX, descartadas debido a que no eran igual de eficientes que Qt o no eran nativas.
- **Flutter:** Framework de Dart utilizado para el frontend de móvil. Elegido por su capacidad de desarrollar una aplicación multiplataforma con el mismo código base. Otra alternativa que fue descartada fue React Native ya que no ofrecía la opción de desarrollo multiplataforma.

- **Docker:** Utilizado para contenerizar y desplegar el backend. Elegido por su eficiencia y simplicidad. Alternativas planteadas y descartadas fueron el uso de máquinas virtuales o de Kubernetes, ambas descartadas por ser demasiado complejas para nuestro objetivo.
- **PostgreSQL:** Utilizada como la base de datos para el backend. Elegida por su capacidad multiplataforma, eficiencia, seguridad y simplicidad de manejo. Las alternativas propuestas a PostgreSQL fueron MySQL, MongoDB; la segunda opción fue descartada ya que preferíamos utilizar un modelo de base de datos relacional y MySQL fue descartado ya que no permitía manejar bases de datos tan grandes como PostgreSQL.

4.2.8. Otros aspectos técnicos de interés

- La **base de datos** implementada será **SQL**. Hemos optado por un RDBMS SQL (PostgreSQL) porque garantiza transacciones ACID para operaciones críticas (estados de partidas), soporta de forma nativa relaciones complejas y consultas agregadas, y se integra perfectamente con Django ORM. Además, el equipo ya domina SQL, lo que acelera el desarrollo y reduce riesgos.
- El tipo de **API será RESTful**. Elegida por su flexibilidad y por ser la más adecuada para aplicaciones modernas. Se planteó utilizar **SOAP**, pero se descartó por no adecuarse al sistema desarrollado.
- En la aplicación habrá algunas **operaciones asíncronas**, como el **chat** entre jugadores o las **solicitudes de amistad**, con el fin de mejorar la experiencia a nivel de usuario.
- **Tipos de aplicaciones**:
  - Aplicación móvil: Nativa para **Android**.
  - Aplicación de escritorio: Nativa para **Linux**.

  Se optó por el desarrollo de aplicaciones nativas ya que nos resultó más interesante que el desarrollo web.
- **Medidas de seguridad**:
  - **Autenticación y Autorización**: Se utilizarán **tokens JWT** (JSON Web Tokens). Los usuarios deberán autenticarse antes de acceder a recursos protegidos.
- **Instalaciones y Despliegues:**
  - **Contenerización con Docker:** El equipo de backend empleará contenedores e imágenes de Docker para el despliegue del servidor y la base de datos.
  - **Servidor síncrono (Gunicorn)**: Se empleará un servidor Gunicorn en la etapa inicial para **peticiones HTTP**.

- **Servidor en Cloud**: En etapas más avanzadas del proyecto, se utilizará un servidor en Cloud (p.e. OVH Cloud), donde se lanzará con Docker el servidor y la base de datos. De esta manera, frontend podrá acceder a la API a través de una dirección común.
- **Servidor Asíncrono (Daphne):** Para gestionar las operaciones asíncronas anteriormente mencionadas.
- **Emuladores y dispositivos físicos**: Durante el desarrollo, los equipos de frontend harán usos de emuladores de Android o del entorno de escritorio. En etapas finales se desplegará con dispositivos físicos reales.

5. Memoria del proyecto

En esta sección se recoge una visión general del desarrollo del proyecto, incluyendo su inicio, la organización interna del equipo, la planificación y el seguimiento de tareas, así como los distintos ajustes realizados a lo largo del proceso. Se describen tanto los aspectos técnicos como los organizativos, reflejando la evolución del trabajo en equipo desde la fase inicial hasta el cierre del proyecto.

5.1. Inicio del proyecto

El inicio del proyecto presentó ciertas dificultades en todos los frentes, principalmente debido a la novedad de las tecnologías y herramientas empleadas. Al principio, el equipo tuvo que adaptarse a un entorno de trabajo que combinaba múltiples tecnologías desconocidas para algunos integrantes, lo que ralentizó el proceso inicial. Sin embargo, a medida que se realizaron tutoriales y pruebas, se logró un mejor dominio de las herramientas y metodologías, permitiendo establecer un flujo de trabajo más eficiente.

En el área de backend, se encontraron varios inconvenientes durante la configuración inicial, especialmente en el uso de Docker para meter en contenedores el servidor y la base de datos. Estas dificultades fueron resueltas progresivamente a través de pruebas iterativas y ajustes en la configuración de los contenedores. A pesar de estos contratiempos iniciales, el equipo logró estabilizar el entorno de desarrollo y desplegar correctamente los servicios requeridos para la fase inicial del proyecto.

Por su parte, el equipo de frontend también enfrentó una curva de aprendizaje pronunciada, particularmente en el uso de emuladores y entornos de desarrollo específicos. No obstante, mediante la realización de pruebas y la exploración de documentación técnica, se adquirió la experiencia necesaria para avanzar con el desarrollo de las interfaces de usuario.

Conforme el equipo se fue familiarizando con las herramientas y metodologías de trabajo, se logró establecer un ritmo de desarrollo más sólido, alineado con los objetivos definidos en la planificación inicial.

5.2. Ejecución y control del proyecto

Una vez superadas las dificultades iniciales, el equipo estableció un flujo de trabajo que combinaba herramientas colaborativas, metodologías ágiles y una comunicación constante. En los siguientes apartados se detalla cómo se organizaron las tareas, cómo se gestionó la comunicación interna, el seguimiento del progreso y los cambios realizados sobre la marcha en cuanto a planificación, herramientas y procesos.

5.2.1. Reparto de trabajo entre los miembros del equipo

A lo largo del proyecto, las *issues* se han ido creando de forma oportunista, es decir, según surgían necesidades, problemas o nuevas funcionalidades que queríamos implementar. No se realizó una planificación cerrada al principio, sino que se fue ajustando dinámicamente conforme avanzaba el desarrollo.

En general, en todos los equipos no hubo problemas significativos en cuanto a la creación o asignación de *issues*. La comunicación fluida entre los miembros fue clave para ello: cuando alguien detectaba un problema o veía una mejora posible, lo comentaba por uno de nuestros canales recurrentes, mencionados en el siguiente apartado, o lo creaba directamente en el repositorio, y se decidía de manera informal quién lo abordaría según la disponibilidad de cada uno en esa semana.

No se estableció un mecanismo formal para repartir las tareas ni se hizo un seguimiento exhaustivo. Sin embargo, gracias a esta dinámica flexible y colaborativa, no se percibieron desequilibrios importantes en la carga de trabajo dentro de los equipos. En todo momento hubo voluntad de colaboración y reparto equitativo.

5.2.2. Comunicación interna

La comunicación interna, ya fuera entre sectores o con el grupo entero, siempre ha sido mediante reuniones en canal de voz o mediante el intercambio de mensajes en *Discord* o *WhatsApp*.

El funcionamiento de los canales de comunicación ha sido óptimo, los canales en el servidor de `Discord` han sido bien utilizados, teniendo uno general, otro para registrar horas y un canal por equipo (backend, escritorio y móvil). De media se tardaba medio día para responder consultas o peticiones de otros miembros del equipo, siempre que se respetase un horario diurno.

5.2.3. Medición del progreso del proyecto y rastreo de cosas completas e incompletas

La evaluación del progreso del proyecto se ha llevado a cabo mediante las *issues* de `Github`.

Al principio se crearon las *issues* y *subissues* de las distintas cosas a completar y, conforme se han ido completando, se han cerrado las distintas *issues* para llevar un seguimiento de las cosas completadas y a completar.

No obstante, al tener tantas *issues*, para tener una idea adecuada del porcentaje de tareas completadas, también nos estamos centrando en los requisitos funcionales completados.

En concreto, en escritorio se completaron **110** *issues*, en móvil se completaron **68** *issues*

y en backend se completaron **38** *issues*.

5.2.4. Ajustes realizados por divergencias en el calendario

Comparando el calendario inicial con las ocurrencias actuales, no se observa una gran diferencia. A grandes rasgos, se ha llevado a cabo adecuadamente lo indicado en el calendario.

No obstante, al final, no solo hemos realizado lo que se mencionaba en el calendario, sino que hemos decidido avanzar con tareas que habíamos acordado que se llevarían a cabo más adelante, como el almacenamiento de las estadísticas o la integración de la lógica en las interfaces, principalmente por tener el tiempo y las herramientas necesarias para poder llevarlas a cabo.

5.2.5. Ajuste en tecnologías y herramientas empleadas

En cuanto a las tecnologías y herramientas, no ha habido que hacer ningún cambio a las planteadas inicialmente.

5.2.6. Control de versiones, construcción y despliegue

El trabajo y control de código se ha llevado a cabo usando git, organizando el código principal en la rama main de los distintos repositorios de *Github*.

Para desarrollar las nuevas funcionalidades se crearon ramas secundarias. Donde se desarrollaban las implementaciones necesarias. Una vez comprobado su correcto funcionamiento, se fusionaron con la rama de main o se desplegaba en el contenedor de Docker en el caso de backend.

En general, no ha habido ningún problema con la integración, construcción o despliegue de las funcionalidades, ni en backend ni en frontend.

Por último, también han sucedido problemas de conflictos debido a un uso mejorable de git, que no causaron muchos daños posteriores.

Para evitar este tipo de problemas, se decidió implementar revisiones más estrictas para asegurarse de que los assets y dependencias estuvieran correctamente gestionados y que las nuevas funcionalidades funcionaran correctamente antes de su subida.

5.2.7. Pruebas del software

Para realizar las pruebas del software se ha usado la idea inicial de implementar pruebas manuales y automáticas a las distintas funcionalidades.

Las pruebas manuales consistieron en ir explorando las distintas interfaces y funcionalidades claves, validando que la experiencia del usuario fuera la esperada y que no se presentaran errores visibles, o el uso de herramientas como curl para la comprobación de los endpoints, entre otras. En cuanto a las pruebas automatizadas, en el frontend de móvil, se han realizado algunos tests unitarios ejecutados en el entorno de desarrollo de Android Studio.

En desktop se han usado tests unitarios automáticos desarrollados en QT y posteriormente tests automáticos que prueban el funcionamiento de los elementos de la interfaz. En el backend, se ha usado la función de tests integrados en Django, junto con tests en las funcionalidades y peticiones de frontend para verificar el correcto funcionamiento de estas (todos se pueden lanzar mediante el uso de un script) , que se ejecutan en el contenedor de Docker.

En el desarrollo BackEnd se han empleado principalmente pruebas unitarias de caja negra. Para cada petición REST, se ha validado su correcto funcionamiento frente a distintos valores de entrada, aplicando la técnica de "particiones de equivalencia". Además, se ha desarrollado una serie de pruebas que simulan el emparejamiento, transcurso y finalización de partidas 1vs1 y 2vs2. A continuación, se incluye una tabla ejemplificativa de clases de equivalencia utilizadas en las pruebas.

Parámetro	Clase de equivalencia válida	Clase de equivalencia no válida
nombre	1) nombre != None 2) len(nombre.strip())> 0 3) Nombre único	9) nombre == None 10) len(nombre.strip()) == 0 11) ya existe usuario con ese nombre
correo	4) correo != None 5) len(correo.strip()) > 0 6) Correo único	12) correo == None 13) len(correo.strip()) == 0 14) ya existe usuario con ese correo
contrasegna	7) contrasegna != None 8) len(contrasegna.strip()) > 0	15) contrasegna = None 16) len(contrasegna.strip()) == 0

Caso	nombre	correo	contrasegna	Resultado	Clases
1	"nom"	"nom@gmail.com"	123456	201	1, 2, 3, 4 5, 6, 7, 8
Casos de prueba para clases de equivalencia no vlidas					
2	"nom"	"nuevoNom@gmail.com"	123456	400	11
3	"nuevoNom"	"nom@gmail.com"	123456	400	14
4	""	" "	123456	400	9, 13
...					

Figura 5.2.1: Clases de equivalencia para Insertar Usuario

Por último, en el frontend de móvil, hemos desarrollado un conjunto de tests de widgets que cubren todas las pantallas clave.

Estos tests emplean el paquete nativo de testing de Flutter (flutter_test) y se ejecutan desde una línea de comandos mediante flutter test.

Con ellos comprobamos la carga inicial de la UI, la validación de formularios, la navegación entre pantallas y la correcta respuesta ante distintos estados (loading, error, datos vacíos o datos cargados).

Pantalla	**Fichero de test**	**Qué comprueba**
welcome_page.dart	welcome_page_test.dart	Correcto renderizado del título, botón "Empezar" y navegación a login/register.
login_page.dart	login_page_test.dart	Validación de campos (email, contraseña); habilitación/deshabilitación del botón; comportamiento de errores.
register_screen.dart	register_screen_test.dart	Validación de contraseñas coincidentes, envío de datos y mensaje de éxito o fallo.
ranking_screen.dart	ranking_screen_test.dart	Muestra del listado de posiciones, cálculo de porcentajes, pull-to-refresh.
profile_screen.dart	profile_screen_test.dart	Renderizado de estadísticas, imagen de perfil, cambio de pestañas mochila.
search_users_screen.dart	search_users_screen_test.dart	Comportamiento del campo de texto, lista de resultados y botones "añadir amigo".
friends_list_screen.dart	friends_list_screen_test.dart	Carga de amigos existentes, manejo de casos "sin amigos" y refresco de la lista.
friends_request_screen.dart	friends_request_screen_test.dart	Vista de solicitudes entrantes, botones "aceptar/denegar" y actualización de la UI al pulsarlos.
friend_chat.dart	friend_chat_test.dart	Envío y recepción de mensajes, scroll automático y casos de error de red.

Gracias a estas pruebas widget podemos asegurar la estabilidad de la interfaz de usuario frente a cambios futuros, detectar regresiones y garantizar que cada pantalla se comporte correctamente bajo diferentes escenarios de datos y navegación.

5.3. Cierre del proyecto

En la propuesta técnica y económica se estimó:

Tamaño	**Esfuerzos (horas)**	**Costes**
Medio	Mínimo: 330 h Probable: 584 h Máximo: 876 h	Mínimo: 7.646,05 € Probable: 13.530,85 € Máximo: 20.298,78 €

Tabla 4: Resumen del análisis inicial

Tras finalizar el proyecto, obtenemos el siguiente resultado:

Tamaño	**Esfuerzos (horas)**	**Costes**
Medio	843.643h	19.798,76€

Tabla 5: Resumen del análisis inicial

Tras comparar la estimación con el resultado, se observa que el análisis inicial subestimaba los esfuerzos necesarios para completarlo. Durante la propuesta se estimó que el esfuerzo más probable era de 584 horas. Sin embargo, se ha realizado un esfuerzo de 843.643 horas; cosa que se acerca mucho más a la estimación máxima. Con el coste ocurre lo mismo, la realidad se acerca mucho más con el coste máximo estimado.

Esta diferencia de esfuerzo y coste se debe principalmente a la poca experiencia estimando el coste de un proyecto de esta magnitud, y en general a subestimar el aprendizaje y uso de tecnologías nuevas.

Tras este análisis, hemos aprendido que el desarrollo de una aplicación, por simple que parezca, requiere de un gran esfuerzo. Además, el proceso previo de planteamiento y diseño del sistema resulta clave para agilizar el proceso de implementación.

Otro punto clave que hemos asimilado gracias a este proyecto ha sido la importancia de la coordinación, incluso para un proyecto de ocho integrantes, la comunicación y gestión de los recursos humanos son indispensables para el desarrollo general del proyecto.

En cuanto a los riesgos previstos, se han presentado cuatro de los cinco riesgos y, en general, las estrategias de mitigación planteadas al inicio del proyecto han funcionado satisfactoriamente, en especial la comunicación continua y la flexibilidad de tareas (importante para gestionar el riesgo Ri2). Afortunadamente, no hemos detectado la aparición de ningún imprevisto destacable.

El uso de nuevas tecnologías requiere dedicación previa y constante, siendo clave para agilizar el desarrollo del proyecto mediante dichas tecnologías y evitar retrasos o errores provocados por la falta de conocimientos previos. El uso de un sistema de control de versiones (Git y Github) ayuda a gestionar cambios, reducir riesgos y mantener organizado el proyecto.

A continuación, se presentan dos tablas y una gráfica, la primera tabla y la gráfica representan el esfuerzo dedicado por integrante en horas; la segunda tabla representa las actividades realizadas y el impacto al proyecto por integrante.

nombre/semana	Diego	Javi S	Javi D	Jose	Oscar	Alberto	Jorge	Alejandro
28-2	2.5	2.5	2.5	2.5	2.5	2.5	2.5	2.5
7-3	15.2	10.83	0	18.75	18.08	0	13.083	6
14-3	11.5	11.75	20.75	19	3.5	10.75	16	5
21-3	16.8	1	4.5	0	20	3.5	6.75	8
28-3	3.5	0	6.5	0	0	0	3	3
5-4	6.5	3.5	0	1	0	0	0	0
12-4	11	3	0	0	4	0	1.5	13
19-4	9.3	10	5	0	4.5	4	21.75	15
26-4	4.6	23.5	4.5	0	21.25	13.5	5	22
3-5	10	19	17	19	2	23	6	13
10-5	15	16	7.5	8	1	9.5	4.75	8
17-5	24.5	4	20.5	31.5	13	20	20	6
24-5	0	4	2.5	0	6.5	4	1.25	0
25-5	5	1	7	0	5	0	5	0

Tabla 6: Horas trabajadas por persona cada semana

Total general de horas trabajadas: 843.643

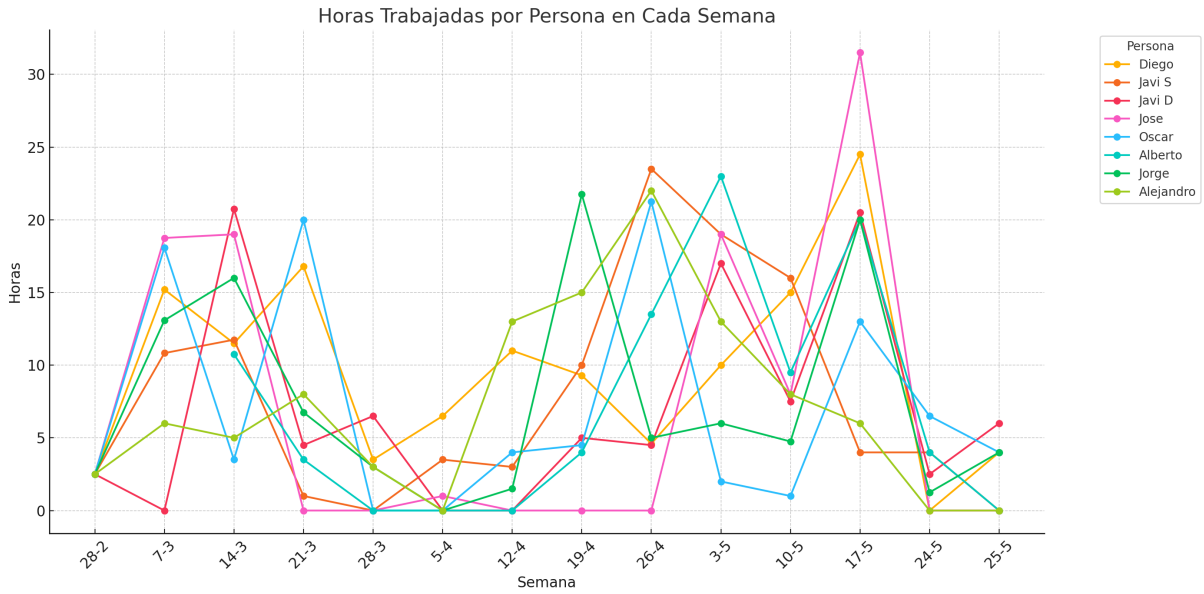


Figura 5.3.1: Gráfica horas dedicadas por desarrollador y semana

Desarrollador	Líneas	Área de trabajo	Actividades realizadas
Jorge Soria Romeo	3771 1950 1512	Backend Móvil Desktop	**BackEnd:** Cuentas de usuario, solicitudes de amistad, partidas, chats y tests. **Móvil:** Pantalla de listado de amigos, de listado de solicitudes de amistad, buscador de usuarios, opción de cambiar de imagen de perfil y chat entre usuarios. **Desktop:** Ayuda en partidas, pantalla que muestra rango del usuario.
Alejandro Villacampa Cabo	3980	Backend	**BackEnd:** Estadísticas de usuario, skins, tapetes, rangos, ayuda en partidas y tests.
Diego Viñuales Bello	3598	Desktop	**Desktop:** Menú Principal, clases auxiliares Icon, ImageButton, Carta y Mano, perfil propio y de amigos, chat de amigos, reconexión, partidas custom, creación de partidas custom, rankings, partidas, settings, baraja base, baraja de poker, baraja paint.
Javier Salafranca Pradilla	3753		**Desktop:** Configuración general del proyecto, iniciar sesión, crear cuenta, pantalla de carga, amigos, rangos, chat de amigos, partidas 1v1, chat en partidas, configuración, cronómetro en partidas, overlay principal de la aplicación, transiciones entre pantallas, logos de la aplicación.
José Manuel Vivas Pozuelo	7247		**Desktop:** tests de escritorio, ajustes
Javier Domingo García	4002	Móvil	**Móvil:** Pantalla de perfil, pantalla de rankings, implementación de las skins, pantalla de rangos, animación de entrada a la app, tests de pantallas, arreglos de errores en distintas pantallas..
Óscar Ventas Fernández	5445		**Móvil:** Crear cuentas, iniciar sesión y pantalla de bienvenida. Pantalla principal de la aplicación, navegación entre la app. Ajustes de la aplicación, añadir música y efectos. Partidas personalizadas, creación de las mismas y conexión a ellas. Pantalla de listar partidas. Diseño GUI Tema de la aplicación y estilos predefinidos. Conexión a partidas Tests unitarios y widgets.
Alberto Serrano Fernández	3031		**Móvil:** Pantalla e implementación de partidas, tanto 1vs1 como 2vs2, chat y ajustes en partida e implementación de listado de partidas disponibles, reconectables y pausadas, perfil de amigos. Arreglo de errores, o mejora de pequeños detalles, en algunas pantallas.
MEDIA	4787		
TOTAL	38289		

Figura 5.3.2: Actividades realizadas por integrante

6. Conclusiones

En líneas generales, las sensaciones acerca del proyecto son muy positivas. El ambiente de trabajo fue muy bueno: nos hemos entendido bien, hemos mantenido un trato cercano y, sobre todo, hemos sido comprensivos cuando alguien no podía aportar más debido a la carga de otras asignaturas. Esta cohesión ha resultado clave para acabar el proyecto con éxito pese a momentos de mayor presión.

Si tuviéramos que reiniciar el proyecto, lo primero que haríamos sería dedicar la primera semana exclusivamente a confeccionar un *road-map* más detallado. En el proyecto actual arrancamos con un esquema demasiado genérico, confiando en que lo afinaríamos sobre la marcha, y eso nos generó algunos picos de estrés cuando aparecieron dependencias imprevistas.

Conservaríamos, prácticamente sin cambios, la forma de comunicarnos. Las reuniones semanales y los canales específicos de `Discord` para cada sub-equipo han funcionado a la perfección y han facilitado un flujo constante de información. También mantendríamos el reparto flexible de tareas. Por último, mantendríamos el uso de control de versiones, lo que nos ha salvado parte del trabajo en más de una ocasión.

A estas mejoras añadiríamos dos iniciativas nuevas. La primera sería establecer una rotación de roles, para evitar que miembros de un equipo tuvieran una idea nula o muy vaga del resto de frentes. La segunda sería aprovechar las reuniones semanales para localizar bugs o defectos menores que en el proyecto actual hemos ido posponiendo o no hemos visto y han causado impedimentos y problemas en la recta final.

En conclusión, repetiríamos la dinámica de comunicación y cooperación que ha funcionado, pero reforzaríamos la planificación detallada y la validación temprana. Con estos ajustes ganaríamos la estabilidad de la que hemos carecido en buena parte de este proyecto y reduciríamos los picos de estrés sin sacrificar la flexibilidad que ha sido, precisamente, la clave de nuestro éxito para completar este proyecto.

A. Anexo I: Riesgos

El impacto se expresa en semanas.

ID	Descripción	Prob.	Impacto	Prob. x Impacto
Ri1	Problemas de organización grupal: los alumnos no están habituados a trabajar en grupos tan grandes.	0.5	2	1
Ri2	Problemas en el tiempo estimado: prácticas de otras asignaturas más largas de lo previsto, controles intermedios o fallos en el código difíciles de identificar pueden alterar el tiempo estimado para una tarea del proyecto.	0.7	2.5	1.75
Ri3	Inexperiencia: los alumnos no han desarrollado proyectos similares, a excepción de en Sistemas de Información.	0.5	1	0.5
Ri4	Paralelización del trabajo ineficiente.	0.5	2.5	1.25
Ri5	No se ha trabajado con las tecnologías elegidas anteriormente.	1	1	1

Tabla 7: Riesgos identificados en el proyecto.

B. Anexo II: Presentación final

En el siguiente anexo se incluyen tanto las diapositivas utilizadas para la presentación pública del proyecto, como las ideas de la teoría utilizadas a la hora de preparar dicha presentación.

B.1. Diapositivas de la presentación final

Figura B.1.1: Diapositiva 1.

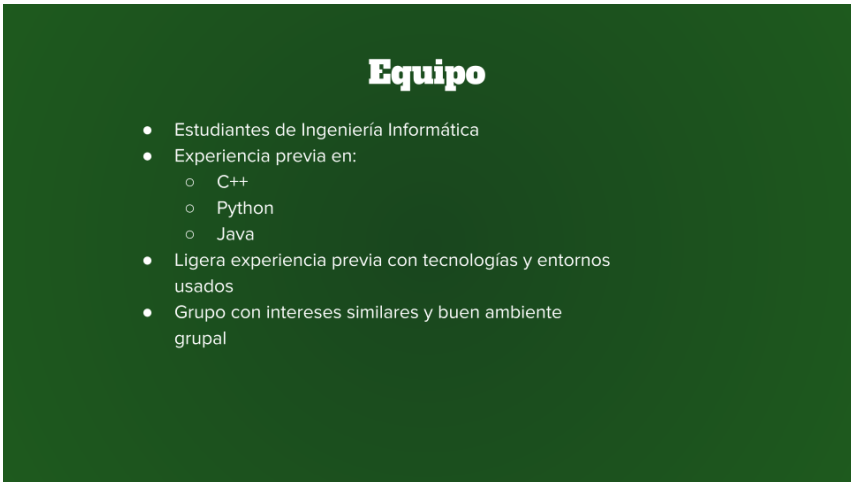


Figura B.1.2: Diapositiva 2.

Figura B.1.3: Diapositiva 3.

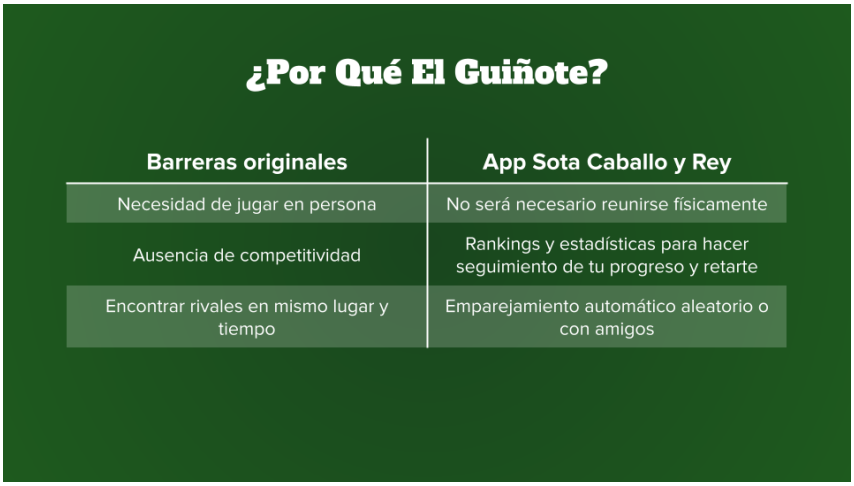


Figura B.1.4: Diapositiva 4.

Figura B.1.5: Diapositiva 5.

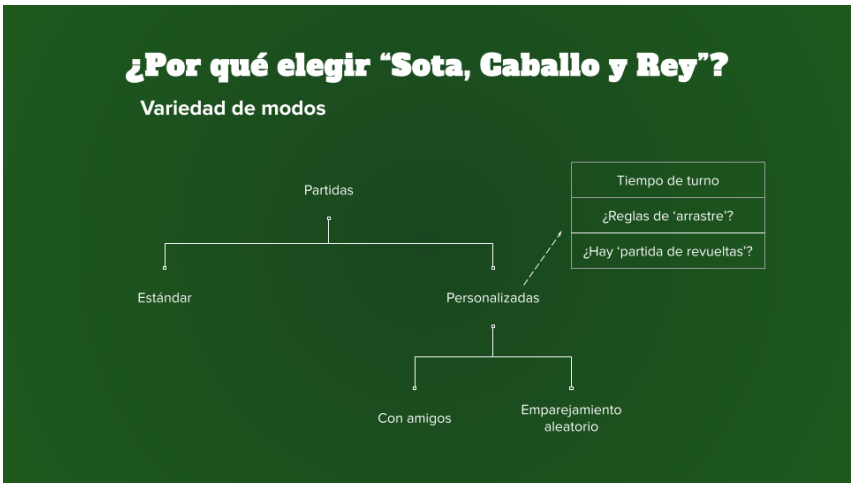


Figura B.1.6: Diapositiva 6.

Figura B.1.7: Diapositiva 7.

Figura B.1.8: Diapositiva 8.

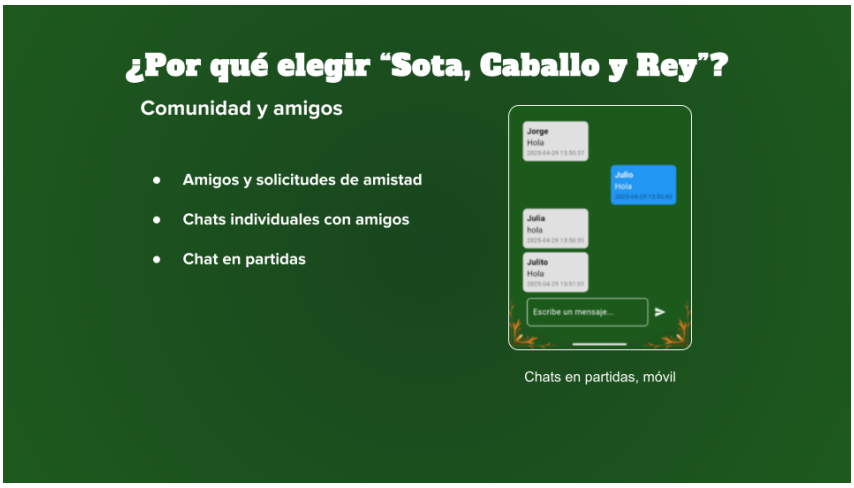


Figura B.1.9: Diapositiva 9.

Figura B.1.10: Diapositiva 10.

Figura B.1.11: Diapositiva 11.

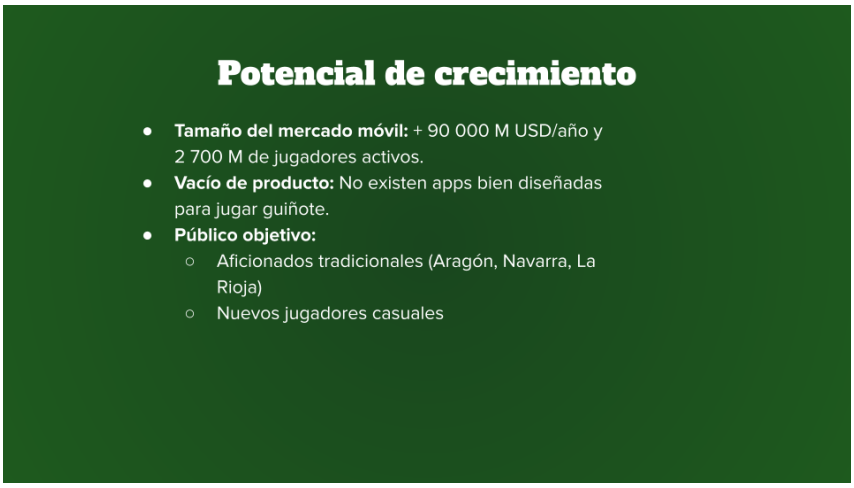


Figura B.1.12: Diapositiva 12.

Figura B.1.13: Diapositiva 13.

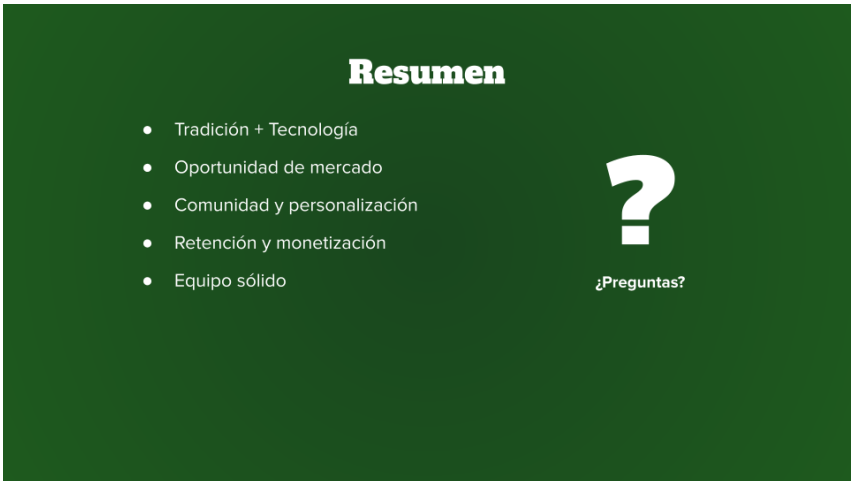


Figura B.1.14: Diapositiva 14.

B.2. Ideas de la teoría tenidas en cuenta

A continuación se presentan, de manera ordenada, todos los conceptos clave utilizados tanto a la hora de crear las diapositivas anteriores, como en el momento de preparar la presentación al público del proyecto.

B.2.1. Conoce y conecta con la audiencia

Al hacer la presentación, nos imaginamos dos tipos de personas en la audiencia:

1. Quienes saben que asisten a una presentación de un juego de cartas.

2. Quienes saben que asisten a la presentación de un nuevo juego de Guiñote.

Para el primer perfil, expusimos nuestra razón para elegir al Guiñote como juego. Para el segundo, explicamos que nos diferencia del resto de las aplicaciones del Guiñote.

B.2.2. La gran idea

Queríamos que la audiencia recordase un mensaje muy claro que representara nuestra aplicación:

"No es solo "Guiñote en una app", sino la experiencia completa y actualizada de Guiñote"

Esta frase fue además nuestro momento S.T.A.R.

Para resaltarla, presentamos el trayecto hasta ella desde el origen, representado por los problemas existentes con los juegos virtuales de Guiñote, hasta nuestro destino, nuestra solución al problema y las características que lo vuelven la solución y la diferencian del resto.

B.2.3. Explicaciones

Desde el inicio intentamos evitar descripciones demasiado literales y, en su lugar, ofrecer guías o informes sobre los conceptos.

Nuestro objetivo era exponer nuestras ideas con palabras claras que permitieran a la audiencia comprender la materia.

Para ello, nos propusimos explicarlo de manera que, si fuésemos parte del público, formulásemos internamente la pregunta: «¿Por qué?».

No pretendíamos demostrar todo lo que sabíamos de la aplicación; solo destacamos los puntos esenciales para la audiencia, como las características principales de sus funcionalidades.

Además, con las introducciones a cada sección, queríamos dar al público una idea clara de su contexto.

B.2.4. Visualización

Para facilitar la comprensión de la información, añadimos herramientas visuales: gráficos estadísticos (el porqué del Guiñote), diagramas de flujo (¿por qué elegir "Sota, Caballo y Rey"?) e imágenes que ilustraban nuestras ideas, como los cambios de humor de los usuarios al usar las aplicaciones existentes y Sota, Caballo y Rey.

B.2.5. Transparencias

En cuanto a las transparencias, dado el tiempo disponible, se intentó añadir alrededor de diez transparencias de un minuto o menos cada una.

Se mantuvo el uso de texto a menos de cien palabras por transparencia, reduciéndolas si era posible para evitar sobrecargar con información al público.

En cuanto a las características más técnicas, los elementos se colocaron de forma que asegurasen un correcto flujo de lectura y unidad, se fomentaron los espacios en blanco especialmente para evitar la congestión de texto, y con los elementos como el fondo, color, imágenes o textos se intentó crear un contraste para permitir la identificación de las ideas.

Y finalmente, NO se añadieron animaciones en las transparencias.

B.2.6. Presentación

A la hora de hablar en público intentamos seguir 3 puntos principales.

1. Evitar el uso de chuletas.
2. Modular la voz y gesticular de manera adecuada.
3. Mantener el contacto visual con la audiencia.

Nuestro objetivo tenía que ser conectar con el público en todo momento y quedarnos con su atención de principio a fin.

C. Anexo III: Resultados revisión intermedia

Metodología del Trabajo y documentación	Respuesta (Sí/No)	Si la respuesta es que no, indicad cómo vais a mejorar en este aspecto. Si la respuesta es que sí, añadid información adicional que consideréis relevante.
Se lleva un registro escrito de las principales decisiones tomadas por el equipo.	Sí	En un canal #actas de discord
Los requisitos definitivos del proyecto han sido revisados por el profesor tutor y cuentan con su visto bueno.	Sí	
Hay un diagrama de navegación con los diseños de las interfaces de los frontends.	No	Tenemos un flowchart muy básico, pero no especifica el diseño de las páginas
Hay un diagrama de Gantt actualizado a fecha de hoy que refleja las fechas de inicio y fin reales de todos los requisitos ya terminados y la planificación de todos los requisitos pendientes.	Sí	Se encuentra en la sección 3.3 (figura 3.3.1). Página 13 de la memoria.
Las issues de GitHub incluyen todas las tareas necesarias para completar el proyecto (incluyendo tests, despliegue, documentación y gestión). Es especialmente crítico asegurarse de que estas tareas cubren el 100% de los requisitos.	No	Están las *issues* correspondientes a la entrega media. Falta por definir alguna *issue* para la entrega final.
Todas las tareas de análisis, diseño, implementación, tests, despliegue y gestión del proyecto tienen un responsable conocido por todos.	No	Cada grupo de trabajo se organiza internamente y divide convenientemente el trabajo. Es su elección elegir o no un responsable.
Se está siguiendo una política de nombrado y gestión de documentos acordada (diseños, código, documentos de gestión, ...)	Sí	
Se está siguiendo alguna guía de estilo de codificación para generar código con un formato homogéneo.	No	
Se está siguiendo alguna guía de diseño de GUI para generar interfaces gráficas homogéneas y usables.	Sí	
Hay un diagrama de arquitectura actualizado con los componentes que forman los frontend y backend del sistema y sus interacciones.	Sí	Todos los diagramas arquitecturales se encuentran en la sección 4.2 de la memoria. Aunque queda pendiente su corrección, tras escuchar los comentarios del profesor.
Hay un documento definiendo la API web que expone el backend.	Sí	https://github.com/UNIZAR-30226-2025-01/guignote-backend/wiki
Hay documentación arquitectural adicional, describiendo otras vistas del sistema (módulos, incluyendo el modelo de datos y despliegue) y aspectos de la dinámica del mismo.	Sí	Existe documentación arquitectural tanto del modelo de datos como de despligue, nuevamente se encuentran todos en la sección 4.2 de la memoria.
Se está usando del sistema de gestión de incidencias de GitHub para asignar todas las tareas de desarrollo.	Sí	
Se está usando un proceso de control de versiones (workflow) que dicta cuándo, y en qué rama se sube código a cada repositorio de GitHub.	Sí	
Se están haciendo pruebas suficientes: unitarias, de integración, de sistema etc. Idealmente al menos una parte de estas pruebas, o todas, son automáticas.	Sí/No	En *backend* se han realizado ficheros de test para las distintas peticiones de nuestra API y un script para automatizar las pruebas. En *frontend,* aunque se han implementado algunos tests unitarios, falta tanto desarrollar tests de integración como automatizar las pruebas.
Se ha desplegado al menos una vez el sistema integrado, en su estado actual, en un entorno similar al que se prevea como entorno de despliegue final (entorno de producción). Idealmente este proceso se ha hecho ya varias veces, e idealmente este proceso se ha automatizado al menos parcialmente.	Sí	Se ha desplegado varias veces durante el desarrollo tanto en emuladores como en dispositivos físicos para ambas plataformas. Además se está empleando una máquina Ubuntu de OVH Cloud.

Estado del proyecto y grado de ejecución	Respuesta (Valor cuantitativo)	Mitigación de posibles problemas
Porcentaje de requisitos funcionales completados.	60,00%	*Si este número es muy bajo, indicad qué vais a hacer para mejorarlo.*
Horas de trabajo realizadas respecto de las previstas del total del proyecto.	189	*Si este número es muy bajo, indicad qué vais a hacer para mejorarlo.*
Estimación del porcentaje del proyecto completado, en desarrollo, y pendiente, en función de las tareas completadas, en desarrollo y pendientes.	30% 10% 60%	*Si el completado y/o el "en desarrollo" es poco, indicad qué vais a hacer para mejorarlo.*
Actividad de cada integrante del equipo desarrollo. Repetid esta sección las veces que haga falta añadiendo el nombre de cada persona del equipo.	**Respuesta (Valor cuantitativo)**	**Si algún integrante tiene grandes desfases respecto al resto, indicad la causa e indicad cómo vais a tratar de reconducir esta situación.**
Diego Viñuales Bello		
Horas dedicadas al proyecto hasta este momento.	30,5	
Horas previstas de dedicación al proyecto en total.	100	
Número de commits realizados en Github.	53	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	1524	
Javier Salafranca Pradilla		
Horas dedicadas al proyecto hasta este momento.	31,2	
Horas previstas de dedicación al proyecto en total.	60	
Número de commits realizados en Github.	30	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	2280	
Javier Domingo García		
Horas dedicadas al proyecto hasta este momento.	34	
Horas previstas de dedicación al proyecto en total.	75	
Número de commits realizados en Github.	15	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	1573	
Óscar Ventas Fernández		
Horas dedicadas al proyecto hasta este momento.	44,33	
Horas previstas de dedicación al proyecto en total.	100	
Número de commits realizados en Github.	31	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	1665	He restado unas 1200 líneas de código debido a que cree el proyecto inicial de flutter y crea bastantes archivos.
Alejandro Villacampa Cabo		
Horas dedicadas al proyecto hasta este momento.	41,75	
Horas previstas de dedicación al proyecto en total.	80	
Número de commits realizados en Github.	17	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	2450	
Alberto Serrano Fernández		
Horas dedicadas al proyecto hasta este momento.	15,5	
Horas previstas de dedicación al proyecto en total.	80	
Número de commits realizados en Github.	12	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	768	
Jorge Soria Romeo		
Horas dedicadas al proyecto hasta este momento.	40,88	
Horas previstas de dedicación al proyecto en total.	80	
Número de commits realizados en Github.	23 + 7 = 30	Backend + Frontend Móvil
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	1653 + 1514 = 3167	Backend + Frontend Móvil
José Manuel Vivas Pozuelo		
Horas dedicadas al proyecto hasta este momento.	40	
Horas previstas de dedicación al proyecto en total.	120	
Número de commits realizados en Github.	43	
Líneas de código vivas en el Github del proyecto (solo aquellas que existen en la última versión del proyecto). No cuentan las bibliotecas, que no deberían estar almacenadas en vuestros repositorios. Para ello podéis usar git-fame tal y como se indica en el guión de la práctica.	2157	